國 立 雲 林 科 技 大 學
National Yunlin University of Science & Technology

資訊管理系畢業實務專題

Norgo：基於 python 的 AI Agent 架構

專題組員：何丞翔 B11123003
李柏達 B11123010
謝佳叡 B11123063

中華民國 114 年 12 月

主目錄

1. 專案執行規劃書
2. 系統需求規格書
3. 系統設計規格書
4. 系統測試報告書

專案執行規格書

目錄

目錄 i
表目錄 iii
圖目錄 iv
第 1 章 專案概論 1
1.1 專案背景 1
1.2 專案動機 1
1.3 專案目標與範疇 2
第 2 章 文獻探討 3
2.1 大型語言模型技術 3
2.1.1 代表性文獻摘要與評析 3
2.1.2 與本專案應用情境的關聯性 4
2.2 檢索增強生成技術 4
2.2.1 代表性文獻摘要與評析 5
2.2.2 與本專案應用情境的關聯性 5
2.3 語音處理技術 6
2.3.1 代表性文獻摘要與評析 6
2.3.2 與本專案應用情境的關聯性 7
2.4 AI Agent 相關技術 8
2.4.1 代表性文獻摘要與評析 8
2.4.2 與本專案應用情境的關聯性 9
第 3 章 專案規劃與時程管理 11
3.1 專案開發方法論 11
3.2 工作分解結構(WBS) 12
3.3 專案時程規劃 14
3.3.1 系統架構與語意處理設計（2024/10 ～ 2024/12） 14
3.3.2 語音互動與記憶檢索功能開發（2024/12 ～ 2025/2） 15
3.3.3 情緒量表與表情動畫設計（2025/3 ～ 2025/4） 16
3.3.4 多平台部署架構實作（2025/5 ～ 2025/8） 16
3.3.5 系統測試與書面撰寫（2025/9 ～ 2025/12） 17
第 4 章 研究展望與建議 18
4.1 研究成果與貢獻 18
4.2 研究限制 19
4.3 未來研究方向 19
4.4 對未來研究與實務應用之建議 20

參考文獻 22

表目錄

表 3.1 WBS Level 0：專案總體 12
表 3.2 WBS Level 1：主要工作領域 12
表 3.3 WBS Level 2：A.系統架構與語意處理設計.....13
表 3.4 WBS Level 2：B.語音互動與記憶檢索功能開發.....13
表 3.5 WBS Level 2：C.情緒量表與表情動畫設計.....13
表 3.6 WBS Level 2：D.多平台部署架構實作.....14
表 3.7 WBS Level 2：E.系統測試與書面撰寫 14

圖目錄

圖 3.1 工作分解結構圖 12
圖 3.2 系統架構與語意處理設計建置甘特圖 15
圖 3.3 語音互動與記憶檢索功能開發甘特圖 15
圖 3.4 情緒量表與表情渲染甘特圖 16
圖 3.5 多平台部署架構實作甘特圖 16
圖 3.6 系統測試與書面撰寫甘特圖 17

第1章 專案概論

本章旨在闡述本專案之發展脈絡與核心價值。首先，說明在生成式人工智慧技術浪潮下，虛擬角色互動模式的變革與現況；接著，剖析當前相關應用在技術開放性上的不足，從而確立本專案開發『具備長期記憶與情感互動能力之輕量化代理人』的核心動機；最後，具體界定本專案之功能範疇、預期目標及其技術邊界，為後續系統開發奠定明確之執行準則。

1.1 專案背景

近年來，隨著生成式人工智慧與虛擬人技術的快速發展，虛擬角色已逐漸具備語音互動、表情控制與即時反應等能力。其中，以 AI VTuber （Virtual YouTuber） 為代表的應用在全球迅速興起，展現了人工智慧與娛樂產業結合的巨大潛力[1]。

以開發者 Vedal 所創作的 Neuro-sama 為具代表性的案例之一。其初始版本以遊戲 *osu!* 畫面為主要輸入，透過模型訓練達成自動化遊戲操作[2]。開發者隨後加入 Live2D 形象，並於 2022 年 12 月 19 日正式以 VTuber 身份在 Twitch 正式展開活動。藉由整合語音辨識、語言推理與動畫控制模組，該系統能在直播情境中進行即時互動、遊戲操作與情緒呈現[3]。此種具自主對話能力的虛擬角色，已超越傳統由人工操控的虛擬偶像模式，形成新型態的 AI 驅動互動架構。

截至 2025 年，Neuro-sama 已在 Twitch 平台上已累積大量追隨者，並於 2025 年 1 月創下 Hype Train 世界紀錄[4]。根據第三方報告與業界分析，其直播內容及周邊收益顯示 AI VTuber 模式具有可觀的經濟潛力，證明結合語音互動與人格塑造的虛擬角色，不僅能吸引長期觀眾參與，亦具備營利可能[5]。

1.2 專案動機

然而，現階段具指標性的 AI VTuber 系統多以商業形式運作，其核心技術並未公開釋出。以 Neuro-sama 為例，其模型架構、語音處理流程與動畫控制機制均屬未開源設計，外界僅能從實況表現與開發者訪談中推測其運作原理。此情形使得相關技術雖備受關注，但缺乏可供學術研究與開發者重現、分析或延伸的參考架構。

在此背景下，本專案旨在探討語音驅動 AI VTuber 的核心互動模式，並評估在現有開放技術與有限資源下，是否能構建具語音互動與基本情緒回應能力的輕量化 AI 代理架構。特別是，Neuro-sama 於長期互動中展現出主題延續、語氣一致與回應自然

度等行為特性，引發對其記憶模組與語境管理機制的研究興趣。如何使 AI 在多輪對話中維持語境連貫性、展現適切回應，並形成類似「長期記憶」的互動模式，是本專案試圖探討的重要議題。

1.3 專案目標與範疇

本專案旨在以 AI Agent 技術為核心，探索語音驅動式虛擬角色互動系統的可行性與人機關係之延展性。研究焦點不僅在於建構具備語音辨識、語音合成與語言生成能力的技術框架，更在於探討人工智慧如何透過語音互動、語意理解與記憶延續機制，營造出近似人際互動的「陪伴感」與「持續性對話體驗」。

為達成上述目標，本專案遵循「低耦合（Low Coupling）」、「高彈性（High Flexibility）」與「模組化（Modularity）」之設計原則，開發出可於多平台運行的 Norgo 系統。整體架構以「輸入—處理—輸出」為核心流程，允許單一輸入對應多組輸出模組，並可依任務需求自由組合推理、檢索與表達層邏輯。開發者僅需初始化核心元件並接入事件監聽器，即可完成跨平台語音互動流程。

本專案之具體目標如下：

1. 重現 AI VTuber 互動核心：
   以 AI Agent 技術模擬 Neuro-sama 之語音互動與對話行為模式，分析其在情緒表達與互動節奏上的特徵，作為研究基準。
2. 語音與語言技術整合：
   結合語音辨識（STT）、語音合成（TTS）與大型語言模型（LLM）推理流程，建立語音驅動的人機互動架構，驗證多模態整合的可行性。
3. 長短期記憶與陪伴感建構：
   導入語意檢索（RAG）結構與向量化記憶庫，模擬 AI 在長期對話中的語境延續能力，並探討此記憶機制如何影響使用者的互動正向回饋。
4. 情緒與表情反饋映射：
   與 VTube Studio（VTS）串接，建立語音與語意情緒特徵對應之表情映射模組，使 AI 能根據語境生成相應表情或動作反應，提升互動的自然度與臨場感。

第2章 文獻探討

本專案之建構涉及多項人工智慧核心技術之跨領域整合。為確立系統架構之學術理據與技術可行性，本章將針對相關領域進行系統性之文獻回顧。內容涵蓋：**大型語言模型（LLM）** 之演進與開源模型選型分析；**檢索增強生成（RAG）** 技術在強化對話記憶與事實一致性上之應用；**語音合成與辨識（TTS/STT）** 之最新發展；以及 **AI Agent 架構** 與虛擬人偶控制之相關研究。透過對上述文獻的評析，將歸納出適用於本專案之技術路線與架構設計依據。

2.1 大型語言模型技術

大型語言模型（LLM）是近年來人工智慧的核心技術之一，透過在海量語料上預訓練的深層神經網路來產生高品質的自然語言回應。2018–2020 年間出現的超大規模模型（如 GPT-3）展示了驚人的跨任務表現，也引發開源與商用兩大類模型路線的發展。商業公司（如 OpenAI、Anthropic、Google 等）推出封閉源代碼的專有模型，參數規模巨大且經過微調，通常在各項基準上領先；然而開源社群在近三年進展迅猛，Meta 等公司開放模型（如 2023 年發布的 LLaMA 系列），學術與開源團隊也陸續推出如 Bloom、Falcon、Mistral 等模型，使得開源 LLM 的性能快速拉近封閉模型的水準[7]。整體而言，LLM 技術從早期著重參數規模擴張，逐漸演進到注重訓練數據品質、對齊人類偏好（RLHF）以及工具調用等能力，並形成商用封閉與開源社群雙軌並進的局面。

2.1.1 代表性文獻摘要與評析

有研究專門比較了開源與封閉 LLM 的差異及趨勢。Chen 等人（2023）指出，封閉模型如 ChatGPT/GPT-4 雖表現強勁，但因訓練細節未公開，存在內容可信度與模型行為透明度的疑慮。例如，封閉模型可能隨著時間更新而性能波動，難以重現先前結果，且曾出現服務中斷、API 呼叫昂貴等問題。相較之下，開源模型具有透明、可自主管理的優勢，可以避開上述限制[6]。Zhou 等人（2024）的研究在醫學文本總結任務上比較開源與商用模型，發現微調後的開源模型（如經 LoRA 微調的 LLaMA-2 或 LongT5）已能接近甚至匹敵商用閉源模型 GPT-3.5 的表現。這證明透過有效的參數高效調適，開源模型有潛力縮小與頂尖商用模型的差距。然而文獻也強調，開源模型的性能提升往往伴隨高昂的計算代價和調校難題。例如，訓練或微調大型開源模型需要大量運算資源，同時須注意避免因微調造成模型原有知識遺忘（災難性遺忘）[7]。總的來說，近年的文獻顯示開源 LLM 在研究社群的協力下進步神速，已在部分

任務上超越封閉 LLM（如 GPT-3.5）；但封閉模型依然憑藉專有資料與架構在綜合能力上保持領先，兩者的比較需視應用場景權衡透明度、成本與性能。

2.1.2 與本專案應用情境的關聯性

本專案旨在打造具持續對話與陪伴能力的 AI VTuber，選擇何種 LLM 作為語言中樞至關重要。開源模型允許團隊對模型進行本地部署與定制，避免商用 API 成本負擔及政策變動風險。同時可透過微調讓模型更貼近 VTuber 角色定位和領域知識（例如角色背景設定、觀眾常問問題），增強對話的連貫性和個性化。然而，開源模型若參數較小，可能生成品質或上下文理解不及業界最先進模型，影響對話體驗。商用大型模型（如 GPT-4）則具備強大的推理與上下文理解能力，在建立擬人對話時效果突出，但其閉源特性帶來透明度與控制權問題，而且即時直播場景下持續調用 API 的成本和延遲也是考量重點。因此，本專案需要在性能與可行性之間取得平衡：文獻顯示透過檢索增強、知識蒐集或少量微調，較小的開源模型也能在特定任務上逼近大型模型效果。這暗示我們可先以開源 LLM 為基礎架構對話代理，配合其他模組提升效果，在預算與資源限制下實現近似人類的對話體驗。同時持續關注最新研究進展，評估開源模型是否足以支撐 VTuber 所需的長時對話、個性塑造等功能，必要時再考慮借助商用模型作混合式架構，以充分利用各自優勢。

2.2 檢索增強生成技術

檢索增強生成（Retrieval-Augmented Generation, RAG）是一種結合資訊檢索與語言生成的混合式架構，旨在補足純生成模型在知識範圍與事實一致性上的侷限。傳統的大型語言模型（Large Language Models, LLMs）基於封閉式參數記憶生成內容，當訓練語料在時效性或領域覆蓋上不足時，容易產生知識錯誤與幻覺現象。RAG 的核心概念在於：模型於生成階段前，先從外部知識庫（如維基百科）中檢索出與輸入查詢相關的文本片段，並將這些檢索結果作為輔助上下文，供生成模型在回答時參考，從而提升內容的事實性與可信度。

Lewis 等人（2020）於 NeurIPS 發表的研究首次系統性提出 RAG 架構[8]，結合 Dense Passage Retriever（DPR）作為檢索器與 BART 序列生成模型（Sequence-to-Sequence Generator）作為生成核心，證實在開放域問答（Open-domain QA）等知識密集型任務中，能顯著提升回答準確率與事實一致性。該研究亦區分了 RAG-Sequence 與 RAG-Token 兩種生成方式，前者以整篇文件為單位整合檢索內容，後者則於逐詞生成階段動態更新檢索資訊，進一步提升生成靈活度[8]。

2.2.1 代表性文獻摘要與評析

隨著 RAG 技術演進，Lewis 等人（2020）之設計著重於靜態文件檢索與一次性生成任務，對於如 AI 虛擬代理般需持續對話與記憶延續的應用場景，仍存在記憶維持與上下文管理上的挑戰。

為補足此限制，Lee 與 Rew（2025）發表於 Applied Sciences 的研究提出一套記憶增強型語言模型架構（Memory-Augmented LLM），專為嵌入大學學習平台中的對話代理所設計。該系統在標準 RAG 架構之外，額外整合短期記憶、長期記憶與時間事件記憶模組，能動態追蹤與儲存對話歷史片段，使代理人能夠參照先前語境產生更連貫、個人化的回應。透過引入記憶模組，生成模型得以在回應時同時依據外部知識與對話記憶進行編碼，提升系統對使用者輸入的語境理解深度與語意一致性。

在實驗設計上，研究團隊使用 BERTScore 與大型語言模型（GPT）作為回應評估者，並於實際 LMS 環境中蒐集使用者回饋。結果顯示，導入記憶模組後的對話代理在語意貼合度、上下文保留程度與用戶親近感等指標上皆顯著優於傳統無記憶架構，且在長輪次互動下能維持更高的回應一致性與陪伴感[9]。該研究明確指出，對話型 AI 中導入記憶型 RAG 架構，不僅能提升生成品質，更能增進使用者與系統之間的情感連結與互動意願。

2.2.2 與本專案應用情境的關聯性

本專案擬參考此 Lee 與 Rew（2025）之記憶增強型語言架構，作為語言理解模組與知識生成核心之參考依據。藉此建構具備長期記憶與動態知識整合能力之 AI VTuber 系統。在實際互動情境中，觀眾所提出之問題涵蓋領域廣泛，且常含即時性或跨場域資訊，倘若僅依賴靜態參數語言模型，生成內容之完整性與時效性將受限。

透過 RAG 架構，代理得以針對使用者輸入（包含即時對話上下文）構建查詢，並自預建知識庫中擷取語意相關資訊，補強生成階段之知識依據。此知識庫可整合角色設定、世界觀、以往互動紀錄等資料，使代理在面對涉及特定事件或人物記憶之詢問時，具備回溯、推斷與回應一致性之能力，進而增強其一致性與陪伴感。

此外，對於新興議題或即時性話題（如新聞事件、流行用語等），代理亦可透過網路資訊檢索機制，實現對話內容之動態更新與知識擴充，維持其對話內容之時效性與社會性連結。為此，本專案將探討語義嵌入與向量相似度技術應用於查詢構建模組，以強化語言生成模組之輸入基礎與回應品質。

2.3 語音處理技術

語音辨識（STT）領域中，傳統系統以隱藏式馬可夫模型（Hidden Markov Model, HMM）結合高斯混合模型（Gaussian Mixture Model, GMM）作為聲學模型的核心架構。HMM 負責刻畫語音的時序變化，GMM 用於擬合聲學特徵的機率分佈，但此種組合在處理高度非線性且高維的語音訊號時表現出統計效率不佳[10]。2010 年代初期深度神經網路（Deep Neural Network, DNN）的引入為語音辨識帶來突破性進展：研究顯示將 DNN 與 HMM 相結合的混合模型（DNN-HMM）相較傳統 GMM-HMM 可顯著降低錯誤率，取得約 16%–23% 的相對錯誤率下降。在 Switchboard 等大型語料上，DNN-HMM 已全面超越傳統系統的表現[11]。此後，卷積神經網路（CNN）與長短期記憶網路（LSTM 等 RNN）相繼應用於聲學模型，利用更大的上下文感知能力進一步提升辨識率。此外，端到端架構開始興起：透過連接主義時間分類（CTC）或注意力機制的編碼器-解碼器模型，現代 STT 系統不再依賴語音單元的強制對齊，直接由聲學特徵輸出文字序列，簡化了系統流程並提高訓練效率。

相較之下，語音合成（TTS）技術的發展同樣經歷了從傳統統計模型到深度學習的躍進。早期 TTS 系統主要採用單位串接（unit concatenation）或參數化合成方法（例如 HMM 為基礎的統計參數語音合成），需要人工設計語言學特徵和拼接庫，產生的語音品質往往不夠自然。隨著機器學習進步，學者開始以神經網路取代傳統聲學模型，例如以深度神經網路預測語音參數，逐步改善合成語音的自然度。WaveNet 的出現是 TTS 發展的重要里程碑：這是一種基於深層卷積神經網路的自迴歸聲碼器，由 DeepMind 團隊於 2016 年提出。WaveNet 可以直接對原始波形採樣點建模並逐點產生語音，在多說明者語音數據上訓練後所生成語音的自然度遠勝以往方法，主觀測試顯示其與真人語音的差距縮減了 50% 以上。WaveNet 證明了利用深度學習模型可大幅提升語音合成品質，並為後續各種神經聲碼器（如 WaveRNN、GAN-TTS 等）奠定基礎[12]。同時，端到端的 TTS 模型也在此期間興起，例如 Tacotron 系列模型以編碼器-解碼器架構直接將文字映射到梅爾頻譜，省略了繁複的前處理和後端聲碼器設計。Tacotron 在美國英語數據集上取得了 3.82 分（滿分 5 分）的平均意見得分（MOS），超越當時業界的參數合成系統，自然度明顯提升[13]。深度學習時代的種種技術（包括神經聲學模型和神經聲碼器）的引入，使現代 TTS 系統無論在音質或靈活性上都遠勝傳統 HMM 時期的系統，標誌著語音合成技術進入全新的發展階段。

2.3.1 代表性文獻摘要與評析

在語音辨識領域中，端到端架中自監督與 Transformer 架構。 近年來，Transformer 結合自監督學習的模型在語音辨識取得了突破性進展。其中具有代表性

的是 Facebook AI 團隊的 wav2vec 2.0 框架。Baevski 等人於 2020 年提出的 wav2vec 2.0 採用卷積神經網路提取語音特徵編碼，透過在潛在表示空間隨機遮罩部分時間步並設計對比學習任務來進行預訓練。該模型在未經標註的大量語音上自我訓練，然後微調於少量標註資料即可達到優異性能。實驗結果顯示，在 LibriSpeech 基準上使用全部 960 小時標註資料微調時，wav2vec 2.0 在測試集達到 1.8%/3.3% 的低字錯率（clean/other）；更令人注目的是，在僅利用 1 小時 標註資料（搭配 5.3 萬小時未標註語音預訓練）的極端低資源情況下，wav2vec 2.0 仍超越先前使用 100 小時 標註資料之最佳系統[14]。這證明透過大規模自監督預訓練可大幅減少對人工標註的依賴，為低資源語言的 ASR 發展開闢了新道路。同時，wav2vec 2.0 的創新在於將 Transformer 引入語音表示學習並結合對比損失，大幅提升了特徵表示的泛化能力。然而，該模型的預訓練需要極為龐大的無標註語料和計算資源，且預訓練後仍需針對下游識別任務進行微調調適，這在一定程度上限制了其在資源受限環境下的直接應用。

在 TTS 領域，最新的研發趨勢致力於將聲學模型與聲碼器融為一體，以進一步提升合成品質與簡化流程。Kim 等人在 2021 年推出的 VITS 模型[17] 即為此方向的代表作。VITS (Variational Inference with adversarial learning for TTS) 將文字至語音的生成任務整合進單一架構中，透過條件變分自編碼器（VAE）結合正規化流（normalizing flow）以及對抗訓練策略，同時學習語音的隱變量表示和高擬真波形生成。相較於以往先產生梅爾頻譜再經聲碼器合成的兩階段方法，VITS 可直接由文字生成語音波形。一方面，模型中的 VAE-Flow 模組負責捕捉文字與語音之間多樣且一對多（one-to-many）的對應關係，並透過隨機時長預測器允許相同文字輸入生成不同韻律節奏的語音；另一方面，對抗式學習引入了多尺度判別器來提升生成語音的逼真度。在單說話人資料集 LJ Speech 上，VITS 達成了與真實人聲幾乎無可區分的 MOS 評分（接近滿分 5 分）[15]。主觀聽測結果顯示，VITS 所合成語音的自然度不僅超越先前最佳的 Tacotron 2 + WaveNet 雙階段系統，甚至與原始錄音難分軒輊。這項研究證明了透過統一的生成模型可以將 TTS 品質推向接近人類的水平，為端到端語音合成樹立了新的里程碑。然而，VITS 的模型結構相對複雜，訓練時需要同時平衡變分學習與對抗損失的穩定性，實現細節較為繁瑣。此外，VITS 目前主要在單一說話人任務上展現卓越表現，在多說話人或風格多樣的合成任務上如何保持同等水準仍有待進一步研究和優化。

2.3.2 與本專案應用情境的關聯性

在 AI VTuber 系統中，語音辨識（Speech-to-Text, STT）與語音合成（Text-to-Speech, TTS）模組直接影響人機互動的流暢度與沉浸體驗。STT 模組的辨識準確率將決定系統對觀眾語音輸入之理解品質，進而影響語言模型的生成結果與互動連貫性。

若語音辨識錯誤頻發，將導致語意偏差與對答錯誤，破壞使用者的即時溝通體驗。因此，本專案必須採用在開放域口語場景中具高準確度與魯棒性的 ASR（Automatic Speech Recognition）模型，並針對 VTuber 互動語境進行語料優化，以適應彈幕用語、口語縮寫與非正式語彙等特性。

另一方面，TTS 模組在虛擬角色的「人格塑造」中扮演關鍵角色。自然且具情感表現力的語音能強化角色形象，提升觀眾對角色的情感連結與陪伴感。本專案因此著重於建立具個性化聲線的 TTS 模型，藉由聲音克隆（Voice Cloning）技術自少量樣本中生成特定音色，達成角色專屬聲音之重建。此外，情緒化語音合成（Emotional TTS）技術亦被納入研究方向，以模擬不同情境下的語調變化，使虛擬角色能依據語意內容與互動情緒動態調整語速、音高與音色，營造更貼近人類交流的表達效果。

綜上，STT 與 TTS 模組分別支撐了本專案在「理解使用者語音指令」與「生成具情感表達之語音回應」兩個層面的互動基礎。透過結合準確的語音辨識與自然的語音合成技術，AI VTuber 系統得以實現高品質的語音互動體驗，為長期陪伴型人工智慧代理的研究奠定關鍵技術基礎。

2.4 AI Agent 相關技術

AI Agent 架構指的是為人工智慧代理設計的系統性模組組合，包括感知、決策、行動等部分，以實現自主的「感知-思考-行動」循環[16]。在本專案情境下，AI VTuber 作為一種虛擬代理，需要整合多種技術（語音、語言、動畫）來產生連貫的對話與互動行為。其架構通常採用事件導向設計（Event-Driven Design），即系統透過事件驅動狀態更新和動作觸發。例如，觀眾發來一條訊息或語音即是一個事件，代理收到事件後進行語義理解與推理，然後根據決策產生響應語句及對應的動畫動作，再透過語音合成和角色渲染輸出。這種事件迴路保證代理能對外部刺激做出及時反應，同時透過內部狀態保持對話上下文的延續。

2.4.1 代表性文獻摘要與評析

雖然目前直接以「AI VTuber」為主題、並完整討論其系統架構的學術研究仍然不多，但在對話代理與虛擬人偶控制等相關子領域，已出現若干與本專案高度相關的工作。其中，Baihaqi 等人（2025）在 Interspeech 2025 發表的研究展示了一個 LLM 驅動的聯動動作控制模型：他們利用大型語言模型分析對話的語句及其預計語音長度，產生一系列原始動作單元（如點頭、揮手等）及各自的持續時間，從而驅動虛擬人偶 ERICA 做出與語音同步的對話手勢。具體而言，LLM 接收當前要說出的語句以及文

本轉語音（TTS）預計的語音時長，輸出若干動作指令，每個指令包含動作類型與時間長度，總體動作時長與語音時長相匹配，以確保口語與肢體動作的協調一致[17]。透過讓 LLM 替代人工編寫的動作規則，該框架能隨對話上下文自適應地生成豐富的非語言行為。相比傳統透過劇本逐句標註表情的方法，LLM 的語意理解能力使代理的動作反應更加多樣且貼合語境。但這種方法的效果取決於提示設計以及模型對動作語義的理解程度。

在實務層面，一些非正式報導與開發者訪談（例如關於 Neuro-sama 等 AI VTuber 的技術分享）指出，現行成功案例多採用模組化與分層式的系統設計，將對話管理、情緒狀態控制、內容過濾與動畫控制等功能拆分為相互協作的子系統，而非單一端到端黑盒模型。這類經驗說明了多模組整合在實務開發中的可行性與必要性，亦顯示學術研究中針對動作控制與對話代理的技術路線，已開始影響實際的 AI VTuber 系統建置。

2.4.2 與本專案應用情境的關聯性

綜合前述 AI Agent 與虛擬人偶相關研究，可以歸納出數項對本專案具有關鍵啟發的設計觀念。首先，事件導向的 AI Agent 架構提供了一個適用於直播與互動情境的系統框架：每一則來自觀眾的訊息或語音輸入皆可視為一個觸發事件，系統需在統一的事件管線中完成語音處理、語意理解與動作決策。這種設計有助於將多模組流程拆解為清晰的階段，並在未來面對高頻率互動或多位觀眾同時參與時，維持系統運作的一致性與可擴充性。

其次，長期對話記憶與角色一貫性是 AI VTuber 之所以能提供「陪伴感」的核心條件。長對話代理與記憶機制相關研究指出，單純依賴模型上下文長度並不足以支撐跨場次的長期互動，必須搭配外部記憶、摘要與檢索等機制，才能在控制資源消耗的前提下保留關鍵情節與用戶資訊。對本專案而言，這意味著在設計 AI VTuber 架構時，需要預先思考如何組織角色背景、歷史事件與觀眾資料，使之得以在後續對話生成中被適當喚回，而非僅停留在單次對談的即時反應。

再者，多模態協調機制是讓虛擬角色「生動感」的重要一環。LLM 驅動動作控制的相關工作顯示，語言模型不僅可生成文字內容，也能作為高階決策器，對表情與動作給出具語境意義的指令。這對 AI VTuber 的啟示在於：語音合成、表情切換與動作呈現不應被視為彼此獨立的後處理步驟，而應整合在同一套決策邏輯之下，確保口語內容、語氣氛圍與視覺表現之間的一致性與連貫感。

綜觀以上，文獻中關於 AI Agent 架構、長期記憶機制與多模態動作控制的研究，為本專案建構 AI VTuber 提供了三項主要指引：

（1）採用事件導向的代理架構來整合語音、語言與動畫模組

（2）在系統設計上預留支援長期記憶與角色一貫性的機制；

（3）將語言決策與視覺表現視為一體化的多模態輸出流程。

在後續系統設計書中，本專案將以此三項觀念作為整體架構與模組劃分的理論基礎。

第3章 專案規劃與時程管理

本章旨在說明本專案之開發方法論、工作分解結構（WBS）以及專案執行時程。首先介紹所採用之專案開發方法論，闡述如何運用敏捷開發框架因應技術不確定性；其次，透過工作分解結構（WBS）將本專案開發任務進行層級化拆解與定義；最後，依據開發邏輯擬定詳細之專案時程規劃，確立各階段之時間軸，以作為專案推進與進度監控之基準。

3.1 專案開發方法論

本專案之開發方法論建立於 Scrum 敏捷開發框架，透過迭代式開發與持續反饋機制，確保系統架構能靈活回應技術挑戰與需求變化。有別於傳統瀑布式流程，Scrum 方法論強調短週期迭代、快速原型驗證與跨功能團隊協作，特別適用於本專案所面臨之多模態技術整合情境。團隊能於各 Sprint 週期中聚焦於特定模組之設計與實作，並透過每次迭代成果驗證技術可行性與效能表現。

在技術選型階段，團隊運用 Scrum 的迭代特性採取實證驅動決策策略。初期 Sprint 中針對各核心模組進行技術預研與效能評估，例如語言模型選型階段分別測試開源模型、蒸餾模型與雲端 API 模型，透過 Benchmark 數據決定最適方案。開發過程中，團隊根據 Sprint Review 結果持續調整技術路線；例如語音合成模組最初採用 Coqui TTS 作為實驗方案，後續因語音腔調需求而導入 BreezyVoice 模型。

於架構設計層面，團隊採取「由內而外」之漸進建構策略。首先實現核心推理流程與基礎資料處理能力，確保系統於任一迭代階段皆能維持可運行狀態，再於後續 Sprint 中逐步擴展語音輸入輸出、記憶檢索與動畫控制等進階功能。此策略有效降低技術風險，並使每次迭代產出之可執行系統可作為下一階段之驗證基礎，形成穩定的持續交付節奏。

系統開發亦遵循 Scrum 的可觀測性與持續改進精神。各模組皆配備完整日誌記錄與測試框架，支援團隊於 Sprint Review 階段進行效能分析與瓶頸診斷。開發過程中，團隊持續追蹤學術與開源社群之最新研究成果，並於 Sprint Planning 階段評估新技術整合可行性。例如記憶模組之設計參考 Lewis 等人（2020）提出的 RAG 架構概念，針對即時互動需求進行輕量化改良，使系統能兼顧回應速度與知識完整性。

綜上所述，Scrum 敏捷開發方法論為本專案提供靈活且務實的開發框架，使團隊能在迭代過程中持續優化架構與功能，確保系統具備可擴展性與穩定性。

3.2 工作分解結構(WBS)

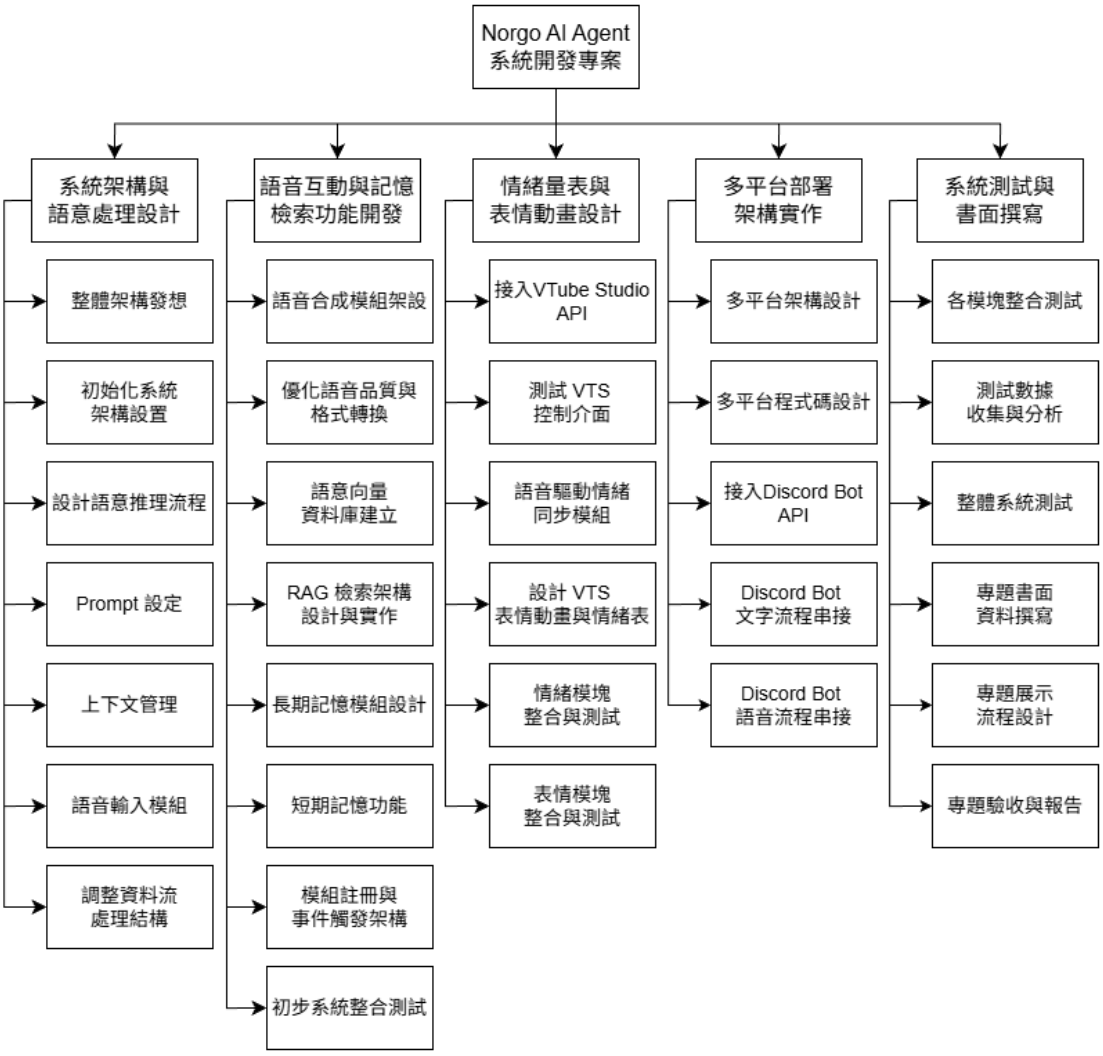

圖 3.1 工作分解結構圖

Level 0：專案總體

表 3.1 WBS Level 0：專案總體

編號	工作包名稱
L0	Norgo AI Agent 系統開發專案

Level 1：主要工作領域

表 3.2 WBS Level 1：主要工作領域

編號	主要工作領域
A	系統架構與語意處理設計
B	語音互動與記憶檢索功能開發

C	情緒量表與表情動畫設計
D	多平台部署架構實作
E	系統測試與書面撰寫

Level 2：工作包

表 3.3 WBS Level 2：A.系統架構與語意處理設計

編號	工作包名稱
A1	整體架構發想
A2	初始化系統架構
A3	設計語意推理流程
A4	prompt 設定
A5	上下文管理
A6	語音輸入模組
A7	調整資料流處理結構

表 3.4 WBS Level 2：B.語音互動與記憶檢索功能開發

編號	工作包名稱
B1	語音合成模組架設
B2	優化語音品質與格式轉換
B3	語意向量資料庫建立
B4	RAG 檢索架構設計與實作
B5	長期記憶模組設計
B6	短期記憶功能
B7	模組註冊與事件觸發架構
B8	初步系統整合測試

表 3.5 WBS Level 2：C.情緒量表與表情動畫設計

編號	工作包名稱
C1	接入 VTube Studio API
C2	測試 VTS 控制介面
C3	語音驅動情緒同步模組
C4	設計 VTS 表情動畫與情緒表
C5	情緒模塊整合與測試
C6	表情模組整合與測試

表 3.6 WBS Level 2：D.多平台部署架構實作

編號	工作包名稱
D1	多平台架構設計
D2	多平台程式碼設計
D3	接入 Discord Bot API
D4	Discord Bot 文字流程串接
D5	Discord Bot 語音流程串接

表 3.7 WBS Level 2：E.系統測試與書面撰寫

編號	工作包名稱
E1	各模塊整合測試
E2	測試數據收集與分析
E3	整體系統測試
E4	專題書面資料撰寫
E5	專題展示流程設計
E6	專題驗收與報告

3.3 專案時程規劃

本專案使用 GitHub 為版本控制與協作平台，實作節奏以實際功能模組為單位逐步推進，並在開發過程中保留完整 commit 記錄以追蹤系統建構脈絡。整體開發歷程可區分為五個主要階段：

3.3.1 系統架構與語意處理設計（2024/10 ～ 2024/12）

專題開發初期聚焦於 Norgo 架構主體與語言模型功能的搭建，主要工作包含：

- 架設本地語言模型推理模組，實作 prompt 設定與上下文管理
- 初步建置語音輸入模組
- 調整資料流處理結構，奠定後續模組串接基礎

此階段完成系統最基本的文字輸入 → 語言模型推理 → 輸出回應的流程雛型，作為後續語音與互動擴展的核心。

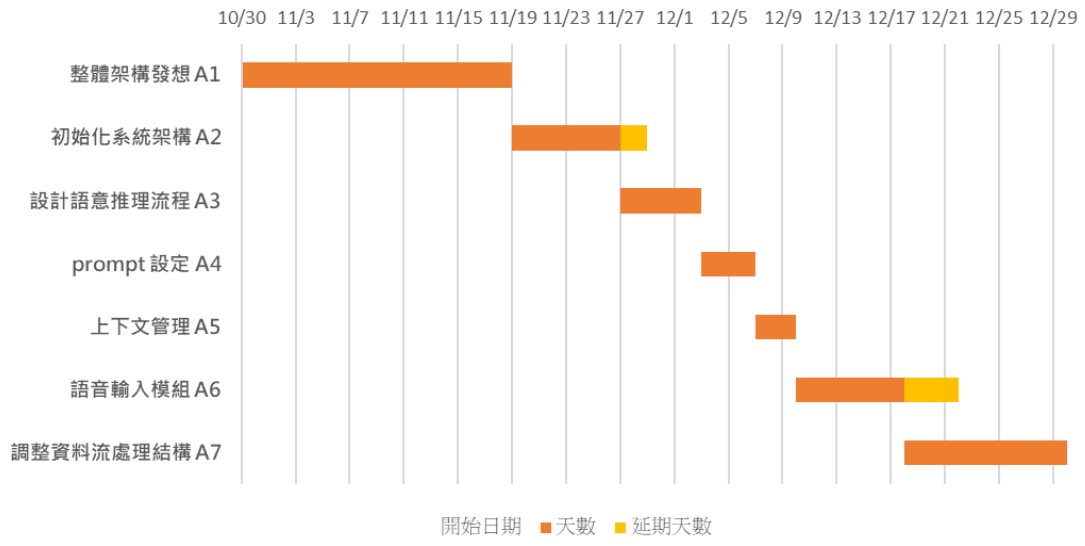

圖 3.2 系統架構與語意處理設計建置甘特圖

3.3.2 語音互動與記憶檢索功能開發（2024/12 ～ 2025/2）

進入中期後，團隊開始實作語音輸出與知識檢索相關功能，重點進度如下：

- 架設語音合成模組，逐步優化輸出品質與語音格式支援
- 建立語意向量資料庫與 RAG 結構，支援知識回補與檢索式問答流程
- 實作模組註冊與 listener 模組，使各元件能以事件觸發方式解耦協作

此階段實作完成後，已實現語音互動 → 知識回應的流程，具備初步語音代理能力。

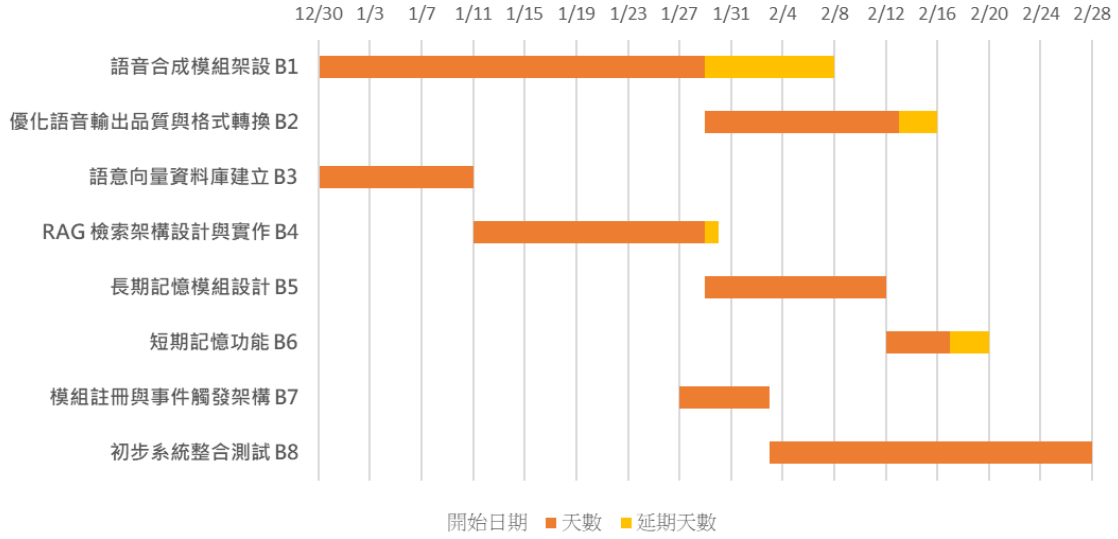

圖 3.3 語音互動與記憶檢索功能開發甘特圖

3.3.3 情緒量表與表情動畫設計（2025/3 ～ 2025/4）

中後期開發集中於應用場景的使用者互動感強化，包含：

- 接入 VTube Studio，實作語音與表情動作的同步控制
- 加入情緒模組，將 Agent 每句話都分類至一種情緒，並修改情緒量表
- 加入表情模塊，根據情緒量表去調整 VTube Studio 的參數變化

此階段實作完成後，系統已實現互動到 Live 2D 表情變化的流程，使 Agent 表現更為生動。

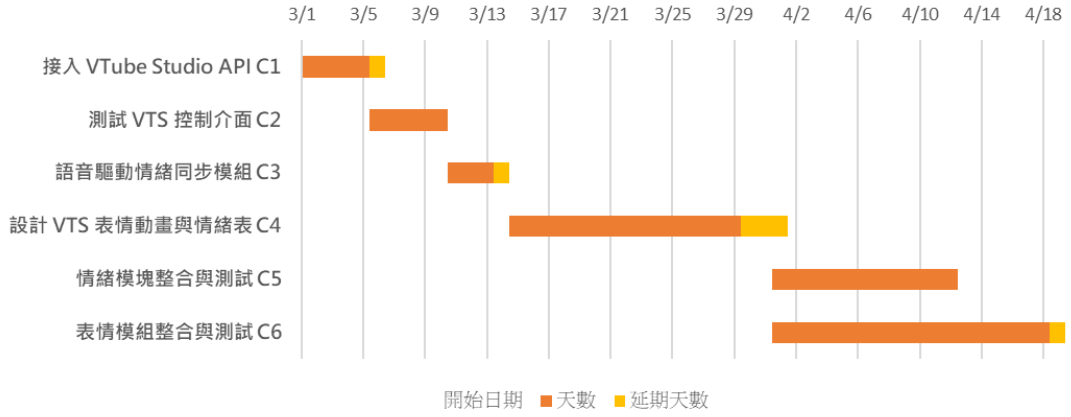

圖 3.4 情緒量表與表情渲染甘特圖

3.3.4 多平台部署架構實作（2025/5 ～ 2025/8）

後期開發集中於平台部署設計與實作，包含：

- 構思多平台部署之架構與程式碼
- 完成 Discord bot 接入，進行完整文字、語音流程測試

此階段整合完成多平台部署基礎，為未來擴展奠定穩固基底。

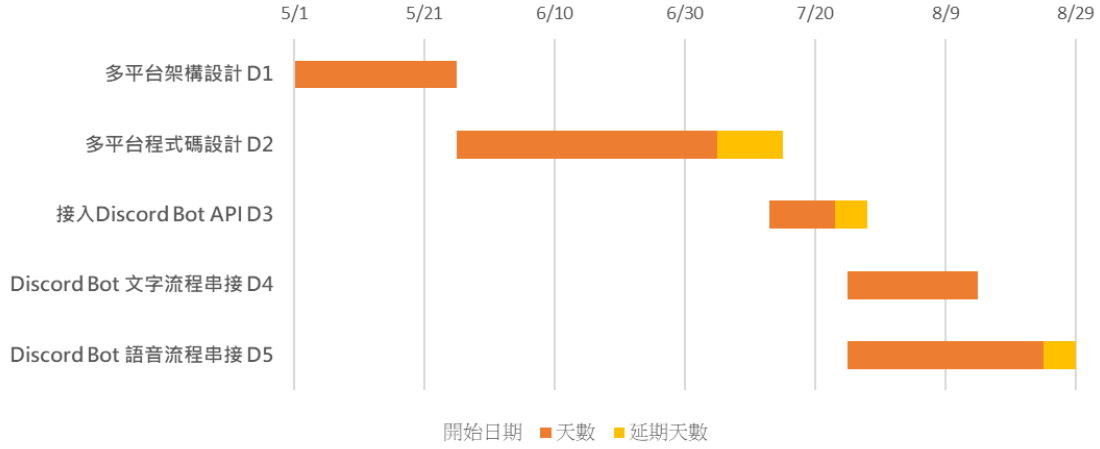

圖 3.5 多平台部署架構實作甘特圖

3.3.5 系統測試與書面撰寫（2025/9 ～ 2025/12）

大後期開發集中於整合測試、系統測試，並完成最終收尾，包含：

- 各模塊整合測試、整體系統測試
- 書面報告、PPT、展示流程設計

此階段完成剩下的測試與數據分析，並設計了一套展示流程，以供驗收使用。

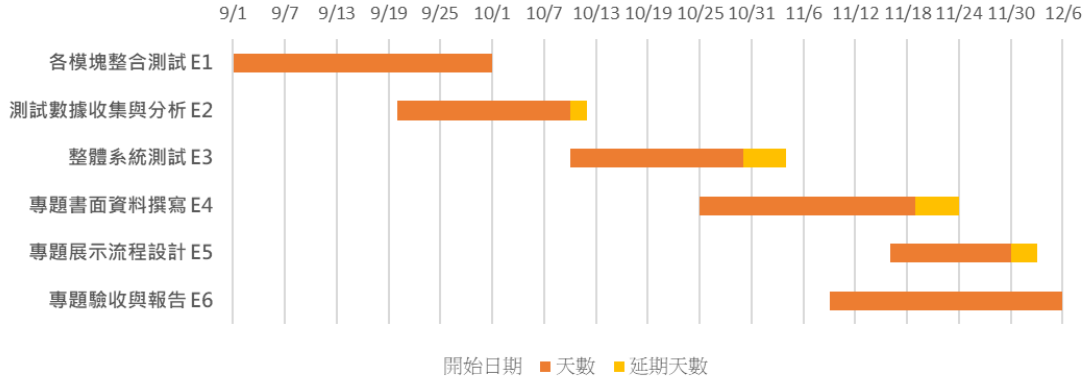

圖 3.6 系統測試與書面撰寫甘特圖

第4章 研究展望與建議

本章旨在呈現本專案之研究成果與未來發展。首先，本章將總結系統在語音互動、記憶機制與表情控制等方面之具體產出與技術貢獻；接著，誠實檢視在硬體資源與技術實作上所面臨之限制與不足之處；隨後，基於當前實作經驗，提出關於多模態整合與系統優化之未來研究方向；最後，針對後續研究者及實務開發者提出具體建議，期能為相關領域之學術研究與應用落地提供參考。

4.1 研究成果與貢獻

本專案建構一套整合語音合成（Text-to-Speech, TTS）、語音辨識（Speech-to-Text, STT）與大型語言模型（Large Language Model, LLM）的語音驅動式 AI Agent 系統，實現由「語音輸入」至「語音輸出」的完整互動流程。透過此一架構，使用者可在無需鍵盤輸入的情境下，與虛擬角色進行自然對話，有效驗證語音導向 AI Agent 在實務應用上的可行性。

在互動體驗設計方面，本專案為 AI Agent 設計了簡易情緒量表，並結合 Live2D 進行表情與動態控制，使虛擬角色得以依據對話情境呈現不同情緒與表情反應，提升角色擬真度與陪伴感。於技術實作層面，考量開發階段硬體資源受限，本專案主要採用雲端運算模式完成推論流程；然研究過程中仍系統性比較多種開源與閉源之語音與語言模型，並整理成比較表，以作為後續模型選型與效能評估之參考。

在語音合成模組方面，本專案針對 BreezyVoice 開源模型之運作方式進行調整與改良，嘗試在既有架構下降低推論延遲，以縮短使用者等待時間。雖最終仍未完全達成理想中的即時回應速度，但相關實驗與調校結果，提供了在有限運算資源下調優語音合成系統之實務經驗。

在檢索增強生成（Retrieval-Augmented Generation, RAG）方面，本專案將其應用於模擬長期記憶機制，並在設計上有別於傳統採用重排序（re-ranking）之作法，提出一套簡化之「仿記憶」檢索流程，聚焦於前四筆高相似度檢索結果（top-4 hit rate）之品質與穩定性，以降低計算開銷並提升實作可行性。此作法雖屬初步嘗試，然對於如何在有限資源下建構具「記憶感」之 AI 對話系統，提供了一可行方向。

在 Live2D 表情與動態控制部分，本專案引入 Perlin Noise（柏林噪音）與 raising 型變化曲線等隨機與平滑控制機制，模擬人類自然微動作與表情細微變化，使虛擬角色在靜止狀態下仍具備生動感，改善傳統固定關節動畫易產生的僵硬感。透過上述設

計，本專案不僅完成一套可運作之語音驅動 AI Agent 原型系統，亦在多模組整合、記憶模擬、表情控制等面向上提出具實務參考價值之設計經驗。

4.2 研究限制

本專案在執行過程中，受限於硬體設備與運算資源，未能在多數實驗條件下採用本地端推論或高規格 GPU 環境，致使部分研究構想僅能停留於設計層面，未能完整驗證。首先，在 LLM 模型調適方面，礙於運算資源有限，無法進行系統化的大規模知識蒸餾或模型微調實驗，故僅能以零樣本（zero-shot）與少樣本（few-shot）提示設計為主，尚無法量化比較不同調適策略對回應品質與延遲時間之影響。

其次，在串流式語音互動流程方面，雖本專案已規劃串流化 TTS、STT 與 LLM 推論之整體架構，但受限於開發人力與時間，本專案仍以較為傳統之「分段式」處理流程為主，未能全面導入端到端串流機制。因此，在極低延遲或高度即時互動之情境下，仍與理想架構存在差距。

在 RAG 與「仿記憶」演算法評估方面，受限於可用語料與資料集規模，本專案尚無法對所提出之檢索策略進行大規模實證驗證，特別是關於是否會因檢索與排序機制設計，而導致特定背景知識被過度強化或偏好召回的問題，仍有待後續研究透過更大規模、多領域資料集加以檢驗。

Live2D 模組方面，由於團隊人力與專業分工限制，無法針對模型底層架構（例如骨架設計、權重配置、物理模擬參數）進行更深入之客製化開發，僅能在既有參數與 API 所提供之控制範圍內進行微調，限制了角色動態表現的上限。

此外，在情緒量表與情緒辨識機制設計上，原始構想為定義約五種情緒傾向，並透過人工標註對話語句之情緒標籤，以訓練一個可推論當前情緒狀態之模型，進而驅動 Agent 情緒演化與表情變化。然而，礙於研究時程與資料標註成本，本專案僅完成概念性設計與部分實作，尚未完成完整的資料標註、模型訓練與量化評估，相關成果因此仍屬初步探索階段。

4.3 未來研究方向

本專案目前仍以語音與文字模態為核心互動管道，然而多模態（multimodal）互動已成為當前生成式模型與 AI Agent 發展之重要趨勢。未來研究可在現有架構基礎上，進一步納入視覺模態，朝向「語音＋文字＋圖像」之整合互動形式發展。

在技術層面上，本專案已關注多模態模型之相關發展，包含以自然語言監督訓練視覺模型的 CLIP 架構，以及結合語言與圖像跨模態理解與生成能力之 PaLI 模型。前者展示了以文字作為監督訊號，使視覺特徵具備高可遷移性與語意對齊能力；後者則顯示多模態模型在圖像敘述、視覺問答與跨語言圖像理解等任務上的潛在優勢。未來若能將此類模型與 Norgo 系統結合，將有助於擴展 AI Agent 在視覺理解與多模態生成領域之應用範疇。

具體而言，未來研究可嘗試導入圖像辨識、圖片敘述（image captioning）或視覺問答（Visual Question Answering, VQA）模組，使系統能支援「語音詢問 → 圖像分析 → 語言回應」之複合式互動流程。例如，在教育場域中，使用者可透過語音詢問教材圖片內容；於客服或智慧助理情境中，系統亦可藉由分析使用者上傳之圖片，提供更具情境感知能力的回應。

此外，在語音互動本身之優化方面，未來可進一步投入以下方向：

（1） 導入真實串流式 STT、TTS 與 LLM 推論流程，降低端到端延遲，提升即時互動品質。
（2） 結合模型壓縮、知識蒸餾與量化技術，以提升開源模型於邊緣裝置或中低階硬體上的部署可行性。
（3） 針對 RAG 與仿記憶機制進行大規模實驗，驗證不同檢索策略對「記憶錯覺」、知識更新與長期對話一致性的影響。

整體而言，現階段成果可視為語音驅動式多模態 AI Agent 之基礎架構雛形，未來研究可在此基礎上逐步擴展模態類型、強化記憶與情緒建模能力，並導入更完整之多模態實驗設計與使用者研究，以提升系統在實務情境中的應用價值。

4.4 對未來研究與實務應用之建議

本專案除提出初步系統架構與實作經驗外，亦對後續研究者與實務導入者提出以下建議，作為未來延伸與應用之參考：

（一）對後續研究者之建議

1. 強化資料與評估設計：建議未來研究在設計 RAG 與記憶機制時，能建立較大且多元之語料庫，並規劃標準化的評估指標（例如：事實正確性、一致性、記憶錯誤比例等），以便系統性比較不同演算法與模型配置。

2. 深化情緒與人格建模：可進一步落實本專案原構想之情緒標註與模型訓練流程，建構具人格與情緒持續性的 Agent，並結合長期對話紀錄，檢視其對使用者黏著度與互動體驗之影響。
3. 探索多模態整合策略：在導入視覺與其他感測模態時，建議同時關注模型層（如 CLIP、PaLI 等多模態模型）與系統層（如模組耦合度、延遲管理與資源配置）的整合策略，以平衡系統複雜度與實際可用性。

（二）對系統開發者與管理者之建議

1. 硬體與雲端資源規劃：在實務部署前，應評估目標場域之預期使用量與延遲要求，事先規劃合適之本地端運算資源與雲端服務，避免因資源不足造成體驗不佳或成本過高。
2. 資料隱私與倫理考量：由於語音與對話資料具有高度敏感性，開發與管理者應建立明確之資料存取、匿名化與留存政策，並確保系統設計符合相關法規與倫理要求。
3. 跨角色協作機制：若系統導入於教育、醫療或客服場域，建議由技術團隊與領域專家（如教師、醫護人員、客服主管等）共同參與需求定義與知識庫建置，以確保系統回應內容之專業性與實務適切性。

（三）對最終使用者與導入單位之建議

1. 逐步導入與試行驗證：建議先以小規模試行或特定情境之示範場域作為起點，收集使用者回饋與操作紀錄，再逐步擴大導入規模，以降低初期風險。
2. 重視使用者教育與介面設計：在實務應用中，應透過清楚的操作引導與友善的介面設計，協助使用者理解系統特性與限制，避免過度期待或誤用系統。
3. 持續性維運與知識更新：為維持系統回應品質與專業可信度，導入單位需預先規劃維運機制與知識庫更新流程，例如定期檢視常見問答、修正錯誤內容與加入新知，以確保 AI Agent 能隨環境與需求演變持續成長。

綜上所述，本專案雖仍存在若干限制，然其在語音驅動 AI Agent 架構、RAG 仿記憶機制以及 Live2D 動態表現等面向，已提供具體的實作經驗與設計參考。期望未來研究與實務導入者能在此基礎上持續深化與擴充，使多模態 AI Agent 得以在教育、醫療與服務等領域發揮更高之應用價值。

參考文獻

[1] C. Xiang, “This virtual Twitch streamer is controlled entirely by AI,” VICE, https://www.vice.com/en/article/this-virtual-twitch-streamer-is-controlled-entirely-by-ai/, Jan. 2023 (accessed Oct. 2025).

[2] MegaMix, “Neuro-sama, the AI VTuber that invaded osu!,” osu! News, https://osu.ppy.sh/home/news/2024-07-01-the-followpoint-neuro-sama-the-ai-vtuber-that-invaded-osu, Jul. 2024 (accessed Oct. 2025).

[3] “Neuro-sama,” Wikipedia, https://en.wikipedia.org/wiki/Neuro-sama, Oct. 2025 (accessed Oct. 2025).

[4] J. Hale, “An AI VTuber is setting world records on Twitch,” Tubefilter, https://www.tubefilter.com/2025/01/02/neurosama-ai-vtuber-twitch/, Jan. 2025 (accessed Oct. 2025).

[5] S. Narita, “AI VTuber Neuro-sama is back from its Twitch ban and acting as strange as ever,” Automaton, https://automaton-media.com/en/news/20230127-17630/, Jan. 2023 (accessed Oct. 2025).

[6] H. Chen, F. Jiao, X. Li, C. Qin, M. Ravaut, R. Zhao, C. Xiong, and S. Joty, “ChatGPT’s one-year anniversary: Are open-source large language models catching up?,” arXiv, arXiv:2311.16989, 2023.

[7] G. Zhang, Q. Jin, Y. Zhou et al., “Closing the gap between open source and commercial large language models for medical evidence summarization,” npj Digit. Med., vol. 7, art. no. 239, 2024, doi: 10.1038/s41746-024-01239-w.

[8] P. Lewis, E. Perez, A. Piktus et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, arXiv:2005.11401.

[9] J. Lee and J. Rew, “Memory-augmented large language model for enhanced chatbot services in university learning management systems,” Appl. Sci., vol. 15, no. 17, art. 9775, 2025, doi: 10.3390/app15179775.

[10] G. Hinton, L. Deng, D. Yu et al., “Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups,” IEEE Signal Process. Mag., vol. 29, no. 6, pp. 82–97, Nov. 2012.

[11] G. E. Dahl, D. Yu, L. Deng, and A. Acero, “Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition,” IEEE Trans. Audio, Speech, Lang. Process., vol. 20, no. 1, pp. 30–42, 2012.

[12] A. van den Oord, S. Dieleman, H. Zen et al., “WaveNet: A generative model for raw audio,” arXiv, arXiv:1609.03499, 2016.

[13] Y. Wang, R. J. Skerry-Ryan, D. Stanton et al., “Tacotron: Towards end-to-end speech synthesis,” in Proc. Interspeech, 2017.
[14] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” in Adv. Neural Inf. Process. Syst. (NeurIPS), 2020.
[15] J. Kim, J. Kong, and J. Son, “Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech,” in Proc. 38th Int. Conf. Mach. Learn. (ICML), vol. 139, Proc. Mach. Learn. Res., 2021, pp. 5530–5540.
[16] T. Masterman, S. Besen, M. Sawtell, and A. Chao, “The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey,” arXiv, arXiv:2404.11584, 2024.
[17] M. Y. Baihaqi, A. F. G. Contreras, S. Kawano, and K. Yoshino, “Co-speech motion for virtual agents in dialogue using LLM-driven primitive action selection,” in Proc. Interspeech, Rotterdam, The Netherlands, Aug. 2025.

專案需求規格書

目錄

目錄 i
圖目錄 iii
第 1 章 需求概述 1
1.1 需求目標與範疇 1
1.2 系統定位 2
1.3 需求限制與假設 3
1.3.1 硬體資源限制 3
1.3.2 即時性要求 3
1.3.3 服務依賴性限制 3
1.3.4 模型相容性差異 4
第 2 章 功能需求 5
2.1 語音輸入處理 5
2.1.1 語音接收與緩衝 5
2.1.2 語音辨識轉換 5
2.1.3 多語言識別與切換 5
2.2 語意理解與推理 6
2.2.1 自然語言解析 6
2.2.2 上下文關聯 6
2.2.3 回應內容生成 6
2.2.4 結構化輸出解析 6
2.3 記憶管理與檢索 7
2.3.1 會話記憶維護 7
2.3.2 語意記憶儲存 7
2.3.3 向量化與索引 7
2.3.4 相似度檢索 8
2.3.5 記憶權重更新 8
2.3.6 記憶生命週期管理 8
2.4 語音輸出合成 8
2.4.1 文字前處理 8
2.4.2 語音合成生成 9
2.4.3 韻律與情緒控制 9
2.4.4 語音品質保證 9
2.5 虛擬角色控制 9

2.5.1 表情狀態管理.....10
2.5.2 動畫參數映射.....10
2.5.3 動作指令觸發.....10
2.5.4 表情過渡平滑.....10
2.5.5 同步協調機制.....11
2.6 平台整合與通訊.....11
2.6.1 事件註冊與分發.....11
2.6.2 模組生命週期管理.....11
2.6.3 訊息格式轉換.....11
2.6.4 平台特定處理.....12
2.7 系統配置與管理.....12
2.7.1 配置載入與驗證.....12
2.7.2 模組動態切換.....12
2.7.3 日誌記錄.....13
2.7.4 錯誤處理與恢復.....13
第 3 章 使用案例.....14
3.1 使用案例概覽.....14
3.2 語音對話互動.....15
3.2.1 前置條件.....15
3.2.2 主要流程.....15
3.2.3 預期結果.....15
3.3 知識檢索問答.....17
3.3.1 前置條件.....17
3.3.2 主要流程.....17
3.3.3 預期結果.....17
3.4 情緒表達展示.....19
3.4.1 前置條件.....19
3.4.2 主要流程.....19
3.4.3 預期結果.....19
3.5 跨平台訊息處理.....21
3.5.1 前置條件.....21
3.5.2 主要流程.....21
3.5.3 預期結果.....21

圖目錄

圖 3.1 Norgo 系統使用案例圖 14
圖 3.2 語音對話互動循序圖 16
圖 3.3 知識檢索問答循序圖 18
圖 3.4 情緒表達展示循序圖 20
圖 3.5 跨平台訊息處理循序圖 22

第1章 需求概述

本章旨在建立 Norgo 系統需求規格書之基礎框架，說明需求制定之目標、系統定位與限制條件。系統需求規格書作為連結系統願景與技術設計之關鍵文件，需明確界定系統應提供之功能能力與應滿足之品質屬性，為後續設計決策、實作驗證與測試驗收提供一致性之基準。

1.1 需求目標與範疇

本系統需求規格書旨在定義 Norgo 系統之功能需求與品質屬性。Norgo 為一基於 Python 開發之 AI Agent 互動系統，整合語音辨識（Speech-to-Text, STT）、大型語言模型（Large Language Model, LLM）、檢索增強生成（Retrieval-Augmented Generation, RAG）、語音合成（Text-to-Speech, TTS）與虛擬主播動畫控制（VTube Studio, VTS）等核心技術，實現多模態人機互動能力。

Norgo 系統之需求目標可歸納為四個核心面向。首先，系統應提供端到端之語音驅動互動能力，使用者可透過自然語音輸入進行對話，並接收具備情感韻律之語音回應，此目標要求系統具備低延遲語音處理能力，並能於對話過程中維持語境連貫性與情緒表達適切性。其次，系統需具備模組化與可擴展架構，各子模組應能獨立封裝與替換，開發者可依據應用場景需求自由選擇本地模型或雲端 API 服務，此需求旨在降低系統耦合度並提升技術迭代效率。第三，系統應建立記憶延續與語境理解機制，透過向量化檢索維持多輪對話之上下文一致性，並能動態擷取歷史對話片段輔助語言模型生成回應。第四，系統需支援情感表達與視覺互動能力，根據語意內容與情緒狀態驅動 Live2D 模型進行即時動畫展演，實現語音與表情之同步互動。

本系統需求規格書涵蓋之範疇包括語音與文字輸入處理、語意理解與回應生成、記憶管理與知識檢索、語音輸出與情感表達、虛擬角色表情與動作控制、平台整合與事件驅動協調六大功能領域。各功能領域之具體需求項目將於第 3 章中詳細定義，包含輸入條件、處理行為、輸出結果與例外處理。此外，本文件亦定義系統應滿足之效能需求、可靠性需求、可用性需求、可維護性需求與安全性需求等非功能需求，此類需求將於第 4 章中完整說明。

1.2 系統定位

Norgo 系統定位為一套語音驅動式虛擬角色 AI Agent 共通架構，核心價值在於提供可重複利用、可擴充之技術基礎，使開發者能快速建置各類語音互動服務。系統以 AI VTuber 之多模態互動型態為主要靈感來源，但不限於娛樂應用，亦可延伸至智慧客服、語音導覽、教育展示等跨領域場景。系統開發之研究背景與動機已於專案執行規劃書第 1 章中完整闡述，技術選型與文獻探討則於專案執行規劃書第 2 章中詳細說明，本文件聚焦於需求層面之定義。

在整體應用生態中，Norgo 系統扮演三項核心角色。第一，系統可作為 AI VTuber 與互動娛樂系統之技術骨幹，支援虛擬直播角色背後的語音與對話核心，透過模組化設計使實況主或內容創作者能依需求替換語音合成音色、調整角色人格設定或導入自訂知識庫。第二，系統可作為跨場域語音互動服務之通用框架，透過事件驅動與平台整合模組支援 Discord、Telegram、LINE 等多種通訊平台，降低導入不同場域之整合成本。第三，系統可作為研究與教學用之開放實驗平台，現有具代表性之 AI VTuber 系統多為封閉實作，本系統以「可說明、可重現」為前提設計，為學術單位、學生專題與實務開發者提供可供實驗 RAG、記憶模型與多模態互動策略之研究場域。

系統之核心能力可概括為「聽得懂、回得上、記得住、表情做得到」四項特質。「聽得懂」指系統能透過語音辨識技術接收使用者語音輸入並轉換為文字，支援多語言識別與背景噪音容忍。「回得上」指系統能透過語言模型理解使用者意圖並生成適切回應，維持對話之邏輯連貫與語境延續。「記得住」指系統能透過記憶管理機制維護短期對話歷史與長期語意知識，使互動具備個性化與陪伴感。「表情做得到」指系統能根據語意內容與情緒狀態驅動虛擬角色展現對應表情與動作，實現語音與視覺之同步互動。此四項能力之技術實作已於系統設計規格書第 3 章中詳細說明，本文件聚焦於此類能力應滿足之功能需求與品質標準。

系統整體流程以「輸入—處理—輸出」為主軸。輸入階段，使用者可透過文字或語音發出請求，系統先進行語音辨識與文字前處理。處理階段，系統結合語意檢索與大型語言模型推理生成回應內容，並解析情緒與動作標記。輸出階段，系統經由語音合成模組產生語音回應，並透過虛擬角色控制模組驅動 Live2D 模型展現對應表情與動作。此流程設計使系統能提供端到端之語音驅動互動體驗，使用者無需理解背後技術細節即可進行自然對話。系統架構之詳細設計已於系統設計規格書第 2 章中呈現。

1.3 需求限制與假設

Norgo 系統之需求制定受到硬體資源、即時性要求、服務依賴性與模型相容性四項主要限制因素之制約。本節說明這些限制如何影響需求範疇界定與優先級設定。

1.3.1 硬體資源限制

系統需於一般消費級硬體環境下運行，此限制直接影響效能需求與功能需求之設定。GPU 記憶體容量與運算能力限制使系統無法要求支援超大型語言模型之完整本地部署，因此功能需求包含雲端 API 與輕量級本地模型之混合支援能力。處理器運算能力限制亦影響記憶檢索功能之複雜度要求，系統採用近似最近鄰搜尋而非精確搜尋，以平衡檢索準確度與回應速度。此類限制使系統需求聚焦於「可於有限資源下穩定運行」而非「追求極致效能」。

1.3.2 即時性要求

語音互動場景對端到端延遲具有嚴格要求，從使用者語音輸入至系統語音回應之總處理時間需控制於數秒內，此限制影響多項功能需求之優先級與設計約束：

- 語音辨識需支援串流處理或快速批次處理模式
- 記憶檢索需避免多階段 reranking 機制
- 語言模型推理需設定生成長度上限
- 表情動畫需預先建立參數映射表

此即時性限制使系統需求強調「低延遲」優於「高準確度」，當兩者衝突時優先滿足回應速度要求。

1.3.3 服務依賴性限制

雲端模型服務涉及 API 呼叫成本與網路連線穩定性，此限制影響系統可靠性需求與部署彈性需求之設定。功能需求包含模型動態切換能力，使系統能依據網路狀態與成本預算自動或手動選擇雲端或本地模型。當雲端服務不可用時，系統需能降級至本地模型繼續運作，而非完全中止服務。此限制使系統需求強調「降級可用」而非「最佳效能」。

1.3.4 模型相容性差異

Live2D 模型之參數結構與反應特性存在個體差異，相同動畫參數於不同模型可能產生非預期之視覺效果。此相容性限制影響虛擬角色控制功能之需求範疇，系統需提供參數正規化與範圍限制機制，但無法保證跨模型之完全一致表現。完整之跨模型適配能力需要存取 Cubism SDK 底層資訊，此需求列入未來擴展方向而非當前必要需求。

上述四項限制因素貫穿整個需求制定過程，影響功能範疇界定、效能指標設定與驗收標準訂定。需求優先級排序時，團隊優先滿足硬體資源與即時性限制，其次考量服務依賴性與模型相容性限制。

第2章 功能需求

本章詳細定義 Norgo 系統應提供之功能能力。功能需求依據系統處理流程劃分為語音輸入處理、語意理解與推理、記憶管理與檢索、語音輸出合成、虛擬角色控制、平台整合與通訊、系統配置與管理七大功能領域。各需求項目描述系統應執行之行為、輸入條件、輸出結果與例外處理，不涉及技術實作細節。

2.1 語音輸入處理

語音輸入處理功能負責接收使用者語音訊號並轉換為文字內容，為系統提供語音互動能力之基礎。此功能需支援即時處理、多語言識別與環境噪音容忍，確保語音輸入之準確性與可靠性。

2.1.1 語音接收與緩衝

系統應能透過麥克風或音訊檔案接收使用者語音輸入。當使用者開始說話時，系統需即時捕捉音訊串流並進行緩衝處理。音訊緩衝區大小應可配置，預設值應能容納至少 5 秒語音內容。系統需支援常見音訊格式，包括 WAV、MP3、OGG 等，並能自動偵測取樣率進行轉換。當音訊輸入裝置不可用時，系統應記錄錯誤訊息並通知使用者檢查硬體設定，而非直接中止運行。

2.1.2 語音辨識轉換

系統應將緩衝區內之語音資料轉換為文字內容。辨識過程需支援串流模式與批次模式兩種處理方式。串流模式適用於即時對話場景，系統持續監聽音訊輸入並於偵測到靜音區間時自動切分語音片段並提交辨識。批次模式適用於離線處理場景，系統一次性接收完整音訊檔案並執行辨識。辨識結果應包含文字內容、語言標籤、時間戳記。

2.1.3 多語言識別與切換

系統應支援中文與英文兩種主要語言之語音辨識。系統可採用自動語言偵測模式或使用者指定語言模式。自動偵測模式下，系統根據語音特徵判斷輸入語言並自動選擇對應辨識模型。使用者指定模式下，系統依據配置參數或使用者指令使用特定語言模型進行辨識。當系統偵測到語言切換時（如中英混雜語音），應能正確識別並標記不同語言片段，而非強制統一為單一語言。多語言支援之具體範圍可依據部署需求擴展，架構設計應預留語言模組擴充空間。

2.2 語意理解與推理

語意理解與推理功能負責解析使用者輸入之文字內容，識別使用者意圖並生成適切回應。此功能為系統核心，需結合語境管理與知識檢索，確保回應內容之邏輯連貫性與語意適切性。

2.2.1 自然語言解析

系統應能解析使用者輸入之自然語言文字，識別句子結構、關鍵實體與語意關係。解析過程需處理口語化表達、語法錯誤與不完整句子，確保系統能理解非正式對話內容。當輸入文字包含特殊符號、表情符號或網路用語時，系統應能正確解讀其語意而非忽略或誤判。解析結果應包含文字正規化版本與語意表示，供後續推理流程使用。

2.2.2 上下文關聯

系統應維持對話上下文，使多輪對話具備邏輯連貫性。系統需記錄最近數輪對話內容，並於生成回應時參考此上下文資訊。當使用者提及代詞（如「它」、「那個」）或省略主語時，系統應能根據上下文推斷所指對象。當使用者詢問與先前話題相關之問題時，系統應能關聯歷史資訊而非將其視為獨立提問。上下文資訊應包含對話角色標籤、文字內容與時間戳記，確保系統能正確區分使用者輸入與系統回應。

2.2.3 回應內容生成

系統應根據意圖識別結果、上下文資訊與檢索知識生成適切回應內容。回應內容需符合對話邏輯，避免答非所問或過度離題。回應語氣應與使用者輸入相協調，當使用者以正式語氣提問時系統應以正式語氣回應，當使用者以輕鬆語氣閒聊時系統可採用較為活潑之表達方式。回應長度應適中，一般情境下建議控制於 50 至 200 字之間，避免過於簡短導致資訊不足或過於冗長導致使用者失去耐心。當系統無法提供有效回應時（如問題超出知識範疇、輸入語意不清），應誠實告知使用者並提供建議性引導，而非生成隨意內容。

2.2.4 結構化輸出解析

系統應能生成包含情緒標記與動作指令之結構化回應。回應內容需包含三個部分：文字內容、情緒標籤與動作標籤。情緒標籤標示回應所表達之情緒類型，如中性、快樂、悲傷、驚訝、生氣、害羞等，每次回應應包含一個主要情緒標籤。動作標籤標示

回應伴隨之肢體動作或特效，如揮手、冒汗、跳躍、思考等，可包含零至多個動作標籤。結構化輸出應遵循預定義之格式規範，確保後續模組能正確解析。當語言模型輸出格式不符合規範時，系統應嘗試自動修正或降級為純文字模式，而非直接拒絕輸出。

2.3 記憶管理與檢索

記憶管理與檢索功能負責儲存對話歷史與領域知識，並於需要時檢索相關資訊輔助回應生成。此功能使系統具備語境延續與知識回溯能力，提升互動之個性化與陪伴感。

2.3.1 會話記憶維護

系統應維護當前會話之短期對話記憶。會話記憶採用佇列結構，儲存最近 N 輪對話之完整內容。當新訊息加入且佇列已滿時，系統應移除最舊訊息以維持固定長度。會話記憶項目應包含角色標籤、文字內容、時間戳記與平台資訊，確保系統能正確追蹤對話脈絡。會話記憶不進行持久化儲存，於對話結束或系統重啟後自動清除。系統應提供會話記憶之查詢介面，使其他模組能存取近期對話內容以維持語境一致性。

2.3.2 語意記憶儲存

系統應建立長期語意記憶庫，儲存歷史對話與領域知識之語意表示。每次對話結束後，系統可選擇性將重要對話片段轉換為語意向量並寫入記憶庫。記憶項目應包含原始文字內容、語意向量、時間戳記、來源對話識別碼與元資料標籤。系統應支援批次匯入功能，使部署者能預先載入領域知識或角色設定資訊。語意記憶庫需支援持久化儲存，確保記憶內容於系統重啟後仍能保留。記憶庫容量應可擴展，當記憶項目數量達到預設上限時，系統應提供記憶清理或歸檔機制。

2.3.3 向量化與索引

系統應將文字內容轉換為固定長度之語意向量，以支援高效相似度檢索。向量化過程需採用預訓練之語意嵌入模型，確保語意相似之文本在向量空間中距離較近。向量維度由嵌入模型決定，系統應能適應不同維度之向量表示。向量索引結構需支援近似最近鄰搜尋，使系統能於毫秒等級時間內完成檢索操作。索引建構過程可於背景執行，避免阻塞主流程。當嵌入模型或索引結構變更時，系統應能重新建構索引而不遺失原始記憶內容。

2.3.4 相似度檢索

系統應根據使用者查詢檢索語意相關之記憶項目。檢索過程首先將查詢文字轉換為語意向量，再於向量索引中搜尋最近鄰項目。檢索結果應按相似度分數排序，分數範圍為 0.0 至 1.0，數值越高表示語意相關性越強。系統應支援可配置之檢索數量參數。檢索結果應包含記憶內容、相似度分數與時間戳記，供回應生成模組參考。當檢索結果數量不足或相似度分數普遍偏低時，系統應能判斷無相關記憶並採用純語言模型推理模式。

2.3.5 記憶權重更新

系統應動態調整記憶項目之選擇權重，避免特定高頻項目持續壟斷檢索結果。每次檢索後，系統根據當前查詢與候選項目之語意相似度，以及項目之歷史引用頻率更新其權重值。權重更新機制需平衡相關性與多樣性，使系統既能檢索最相關記憶，又能避免過度重複。權重計算應考量時間衰減因素，使較舊之記憶項目逐漸降低優先級。系統應支援手動重置權重功能，使部署者能清除權重累積效應並恢復初始狀態。

2.3.6 記憶生命週期管理

系統應提供記憶項目之生命週期管理機制。記憶項目應包含最後存取時間與存取次數等統計資訊，系統可依據此類資訊判斷記憶之活躍程度。當記憶項目長期未被存取且權重值低於閾值時，系統可將其標記為待刪除並於下次持久化時移除，釋放儲存空間並維持記憶庫之語意新鮮度。系統應提供手動記憶管理介面，使部署者能新增、編輯、刪除或匯出記憶項目。記憶匯出功能需支援標準格式，便於備份與跨系統遷移。

2.4 語音輸出合成

語音輸出合成功能負責將系統生成之文字回應轉換為語音，並根據情緒標記調整韻律與語調。此功能使系統能提供自然流暢之語音互動體驗，提升使用者沉浸感。

2.4.1 文字前處理

系統應對待合成之文字內容進行前處理，確保輸入符合語音合成模型要求。前處理包括數字轉換（將阿拉伯數字轉為中文數字）、英文字母處理（決定逐字母唸讀或整詞唸讀）、標點符號轉換（將特殊符號轉為停頓或省略）與文字正規化（移除不可發音之控制字元）。當文字內容包含 URL、電子郵件或程式碼片段時，系統應能識別並採

用適當唸讀策略，避免逐字母唸讀造成之冗長與不自然。前處理結果應保留原始文字與處理後文字，便於除錯與品質追蹤。

2.4.2 語音合成生成

系統應將前處理後之文字轉換為語音音訊。語音合成需支援同步模式與非同步模式兩種執行方式。同步模式適用於對延遲要求不高之場景，系統阻塞執行合成並一次性返回完整音訊。非同步模式適用於即時互動場景，系統以協程形式執行合成並可選擇性支援串流輸出，逐片段返回音訊資料以降低首字延遲。合成音訊應符合標準格式。系統應能根據硬體能力自動調整音訊品質，當運算資源不足時降級至較低品質以確保流暢性。

2.4.3 韻律與情緒控制

系統應根據回應內容之情緒標記調整語音韻律。韻律控制包括語速、音調、音量與情緒強度四個面向。系統應為每種情緒類型預定義對應之韻律參數組合，如快樂情緒採用較快語速與較高音調、悲傷情緒採用較慢語速與較低音調。韻律參數應可透過配置調整，使部署者能微調語音表現以符合角色設定。當文字內容包含強調詞彙或感嘆詞時，系統應能識別並增強該部分之韻律變化，提升表達力。系統應確保韻律調整不影響語音可理解度，避免過度誇張導致使用者難以聽懂。

2.4.4 語音品質保證

系統應確保合成語音之品質符合基本標準。語音輸出需清晰可辨、韻律自然、無明顯機械感或失真現象。當語音合成失敗或輸出品質異常時（如包含雜音、截斷、空白片段），系統應偵測並重新嘗試合成，或降級至較簡單之合成策略。系統應記錄合成失敗之案例與原因，便於後續分析與改進。語音輸出應支援音量正規化，確保不同回應間之音量一致性，避免忽大忽小影響使用者體驗。

2.5 虛擬角色控制

虛擬角色控制功能負責驅動 Live2D 虛擬角色展現表情與動作，根據回應內容之情緒與動作標記實現視覺互動。此功能使系統能提供多模態互動體驗，提升沉浸感與表現力。

2.5.1 表情狀態管理

系統應維護虛擬角色當前之表情狀態，包括臉部表情、眼神方向、眉毛位置與嘴型變化等動畫參數。表情狀態以參數集合表示，每個參數對應 Live2D 模型之一個可控維度。系統應記錄當前表情狀態與目標表情狀態，並計算兩者間之過渡路徑。當情緒標記變更時，系統根據新情緒類型查詢對應之目標參數配置，並啟動過渡動畫。表情狀態管理需考量多參數之同步變化，確保不同參數以協調方式過渡，避免部分參數已完成過渡而其他參數仍停留於初始狀態之不一致現象。

2.5.2 動畫參數映射

系統應建立情緒類型至動畫參數之映射關係。映射表定義每種情緒對應之臉部轉動角度、眼睛開合度、眉毛高度、嘴型變化等參數數值。映射表應支援外部配置，使部署者能依據不同 Live2D 模型調整參數數值以達到最佳視覺效果。當系統支援之情緒類型擴展時，映射表應能輕易新增對應項目而不需修改核心程式碼。映射表除定義靜態參數值外，亦應支援動態參數調整，如為快樂情緒添加隨機微笑幅度變化、為驚訝情緒添加眼神抖動效果，提升表情之生動性與自然度。

2.5.3 動作指令觸發

系統應根據動作標籤觸發對應之動畫效果與特效展示。動作觸發透過 VTube Studio 之 Hotkey 機制實現，系統於初始化階段掃描並註冊所有可用 Hotkey，建立動作指令與 Hotkey 識別碼之對應表。當回應內容包含動作標籤時，系統查詢對應表並觸發相應 Hotkey，驅動 Live2D 模型展現預設動畫。系統應支援單次回應觸發多個動作，並控制動作間之時序關係，避免動作重疊或衝突。當目標 Hotkey 不存在或觸發失敗時，系統應記錄錯誤但不中斷整體互動流程，確保語音回應正常輸出。

2.5.4 表情過渡平滑

系統應實現表情狀態間之平滑過渡，避免突兀之參數跳變。過渡機制採用線性插值或緩動函數計算中間狀態，使動畫變化具備啟動與收尾之自然感。系統應支援多種緩動函數類型，如線性、二次緩入緩出、三次緩入緩出等，並可為不同情緒類型指定適當之緩動曲線。過渡時長應可配置，確保過渡既不過於緩慢顯得遲鈍，亦不過於快速導致視覺衝擊。當多個參數同時過渡時，系統應確保所有參數以相同時長完成變化，維持視覺協調性。

2.5.5 同步協調機制

系統應協調語音輸出與表情動畫之時序關係，使聲音與視覺表現同步呈現。當語音開始播放時，表情應同步切換至對應情緒狀態。當語音包含情緒轉折時（如前半段平靜、後半段激動），表情亦應隨之調整而非維持初始狀態。系統需處理語音合成延遲與動畫過渡時間之差異，必要時調整動畫啟動時機以維持同步感。當系統負載過高導致動畫幀率下降時，應優先保證語音流暢性，表情更新可適度降頻但不應完全停止。

2.6 平台整合與通訊

平台整合與通訊功能負責系統與外部通訊平台之介接，以及內部模組間之事件協調。此功能使系統能接入多種訊息平台，並確保模組間低耦合協作。

2.6.1 事件註冊與分發

系統應提供事件驅動之模組通訊機制。各模組透過註冊事件監聽器訂閱感興趣之事件類型，當事件發布時所有註冊該事件之處理函式被非同步呼叫。事件類型包括使用者訊息接收、語音辨識完成、記憶檢索完成、回應生成完成、語音合成完成、情緒狀態變更等。事件物件應包含事件類型標籤、時間戳記與事件資料字典，確保處理函式能取得必要資訊。系統應支援事件優先級設定，高優先級處理函式優先執行，同優先級者按註冊順序執行。當某處理函式執行失敗時，系統應捕捉異常並記錄錯誤，但不中斷其他處理函式之執行。

2.6.2 模組生命週期管理

系統應管理各功能模組之生命週期，包括初始化、啟動、暫停、恢復與關閉五個階段。初始化階段載入模組配置並建立必要資源，啟動階段使模組進入可接收事件之運行狀態。暫停階段停止模組處理新事件但保留當前狀態，恢復階段使暫停之模組重新開始處理。關閉階段釋放模組佔用之資源並完成清理工作。系統應確保模組啟動與關閉之順序正確，避免依賴關係導致之啟動失敗或資源洩漏。當某模組初始化失敗時，系統應能決定是否繼續啟動其他模組或完全終止，此決策依據模組之必要性等級。

2.6.3 訊息格式轉換

系統應能處理不同平台之訊息格式差異。各通訊平台（如 Discord、Telegram、LINE）具有不同之訊息結構、使用者識別方式與內容類型支援。平台介面模組負責將

平台特定之訊息格式轉換為系統內部統一之訊息物件，使核心處理邏輯不受平台差異影響。統一訊息物件應包含使用者識別碼、文字內容、時間戳記、平台標籤與擴展資訊字典，後者可儲存平台特定之元資料。系統回應亦需經由平台介面模組轉換為平台特定格式後發送，此過程包括文字格式調整、嵌入式內容組裝與附件處理。

2.6.4 平台特定處理

系統應支援平台特定功能之處理，如 Discord 之語音頻道連接、Telegram 之內聯鍵盤、LINE 之快速回覆按鈕等。平台介面模組封裝此類特定功能，並提供統一之抽象介面供核心邏輯呼叫。當核心邏輯需要使用平台特定功能時，應先檢查當前平台是否支援，若不支援則降級至通用功能或跳過該操作。系統應記錄平台能力差異，便於部署者了解不同平台下之功能表現。當新增平台支援時，僅需實作對應之平台介面模組並註冊至系統，無需修改核心處理邏輯。

2.7 系統配置與管理

系統配置與管理功能負責系統參數之載入、驗證與動態調整，以及運行狀態之監控與錯誤處理。此功能使系統具備部署彈性與運維友善性。

2.7.1 配置載入與驗證

系統應支援多層次配置管理機制。預設配置參數內建於程式碼常數，提供開箱即用之基礎設定。環境特定配置透過外部檔案（如 YAML 或 JSON 格式）覆寫預設值，使部署者能調整系統行為而不修改程式碼。敏感資訊（如 API 金鑰、資料庫密碼）透過環境變數注入，避免配置檔案洩露機密資訊。系統啟動時應載入並合併所有層次之配置，並進行格式驗證與範圍檢查。當配置參數格式錯誤、缺少必要項目或數值超出合理範圍時，系統應記錄錯誤並拒絕啟動，而非使用錯誤配置繼續運行。

2.7.2 模組動態切換

系統應支援語音模型、語言模型與平台介面之動態切換。切換機制透過工廠模式實現，系統根據配置參數選擇對應實作類別並建構實例。當配置變更時（如從雲端 API 模型切換至本地模型），系統應能在不重啟之情況下完成切換。切換過程需確保當前處理中之請求能正常完成，新請求自動使用新配置之模組。系統應記錄模組切換事件與原因，便於追蹤配置變更歷史。當目標模組初始化失敗時，系統應能回退至先前配置或使用備援模組，確保服務持續可用。

2.7.3 日誌記錄

系統應提供分級日誌記錄機制，支援 DEBUG、INFO、WARNING、ERROR 四個級別。DEBUG 級別記錄詳細之執行流程與變數狀態，用於開發除錯。INFO 級別記錄重要之系統事件與狀態變更，如模組啟動、使用者請求處理、配置變更等。WARNING 級別記錄異常但不致命之情況，如 API 請求逾時、檢索結果為空、配置參數不理想等。ERROR 級別記錄導致功能失效之嚴重錯誤，如模組初始化失敗、資料庫連線中斷、必要資源不可用等。日誌輸出應包含時間戳記、級別標籤、模組名稱與詳細訊息，並支援輸出至標準輸出與檔案雙重目標。日誌檔案應支援自動輪替，避免單一檔案過大影響讀取效率。

2.7.4 錯誤處理與恢復

系統應具備完善之錯誤處理與恢復能力。當模組執行過程中發生異常時，系統應捕捉異常並記錄詳細之錯誤資訊，包括異常類型、錯誤訊息、堆疊追蹤與觸發條件。系統應根據錯誤嚴重程度決定處理策略。暫時性錯誤（如網路逾時、服務暫時不可用）應自動重試，重試次數與間隔可配置。永久性錯誤（如配置錯誤、資源不存在）應立即終止當前操作並通知使用者。當核心模組發生致命錯誤時，系統應能緊急關閉，完成必要之清理工作（如儲存記憶、釋放資源）後再終止運行。系統應提供錯誤恢復機制，如重新載入配置、重建損壞之資料結構、重新連線外部服務等，使系統能從錯誤狀態自動恢復至正常狀態。

第3章 使用案例

本章透過典型使用情境展示 Norgo 系統之互動方式與功能整合。使用案例從使用者視角描述系統如何被使用，串接第 2 章定義之功能需求，驗證需求完整性與可實現性。

3.1 使用案例概覽

Norgo 系統支援文字與語音兩種輸入模式，並可整合虛擬角色視覺互動與跨平台訊息處理能力。系統之典型使用情境包括語音對話互動、知識檢索問答、情緒表達展示與跨平台訊息處理四類。各使用案例可獨立執行或組合使用，形成完整之多模態互動體驗。

系統使用案例圖如圖 3.1 所示，主要參與者包括一般使用者、系統管理者與外部平台。一般使用者透過語音或文字與系統進行對話，系統根據輸入內容提供回應並驅動虛擬角色展現對應表情。系統管理者負責系統配置、模型切換與記憶管理。外部平台（如 Discord、Telegram）作為訊息傳遞媒介，使系統能接入既有通訊生態。

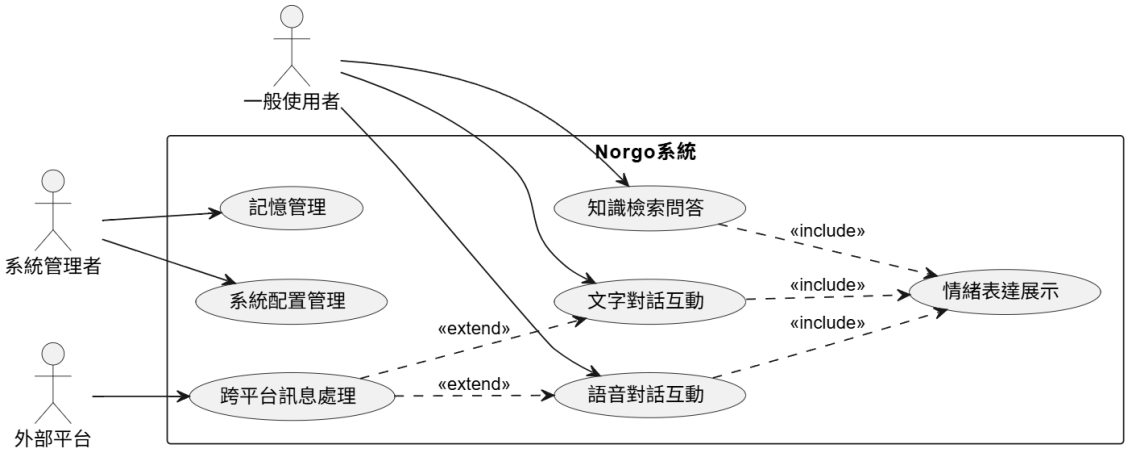

圖 3.1 Norgo 系統使用案例圖

3.2 語音對話互動

圖 3.2 為語音對話互動之循序圖，展示使用者透過語音與系統互動時，各模組間之協作流程與資料傳遞時序。使用者透過麥克風以自然語音與虛擬角色進行對話，系統辨識語音內容、理解意圖、生成回應並以語音形式輸出，同時驅動虛擬角色展現對應表情與動作。此使用案例展示系統之端到端語音互動能力，為 Norgo 系統之核心應用情境。

3.2.1 前置條件

系統已完成啟動並載入必要模型。音訊輸入裝置（麥克風）與輸出裝置（喇叭或耳機）正常運作。VTube Studio 軟體已啟動並載入 Live2D 虛擬角色模型。使用者位於適當之語音互動環境，背景噪音處於可接受範圍。

3.2.2 主要流程

使用者開始對著麥克風說話，如「今天天氣如何？」。系統即時捕捉音訊串流並於偵測到靜音區間後啟動語音辨識，將語音轉換為文字「今天天氣如何？」。系統將辨識結果送入語意理解模組，識別使用者意圖為天氣查詢。系統檢索記憶庫確認是否存在相關上下文，並結合使用者輸入生成提示送入語言模型。

語言模型生成回應內容，如「今天是晴天，氣溫大約 25 度，非常適合出門活動呢！」，並附帶情緒標記「快樂」與動作標記「揮手」。系統將回應文字送入語音合成模組，根據「快樂」情緒調整語速與音調，生成自然之語音輸出。同時，虛擬角色控制模組根據情緒標記調整臉部表情為微笑狀態，並觸發揮手動作。

使用者聽到語音回應並看到虛擬角色展現愉悅表情與揮手動作，完成一次完整之語音互動循環。系統將此輪對話記錄至會話記憶，並可選擇性寫入長期記憶庫供後續檢索使用。

3.2.3 預期結果

使用者感受到流暢之語音互動體驗，系統回應內容與使用者提問相關且語意連貫。語音輸出自然流暢，情緒韻律與回應內容相符。虛擬角色表情變化與語音內容同步，動作觸發時機適當。整個互動過程延遲控制在可接受範圍內，不產生明顯停頓或等待感。

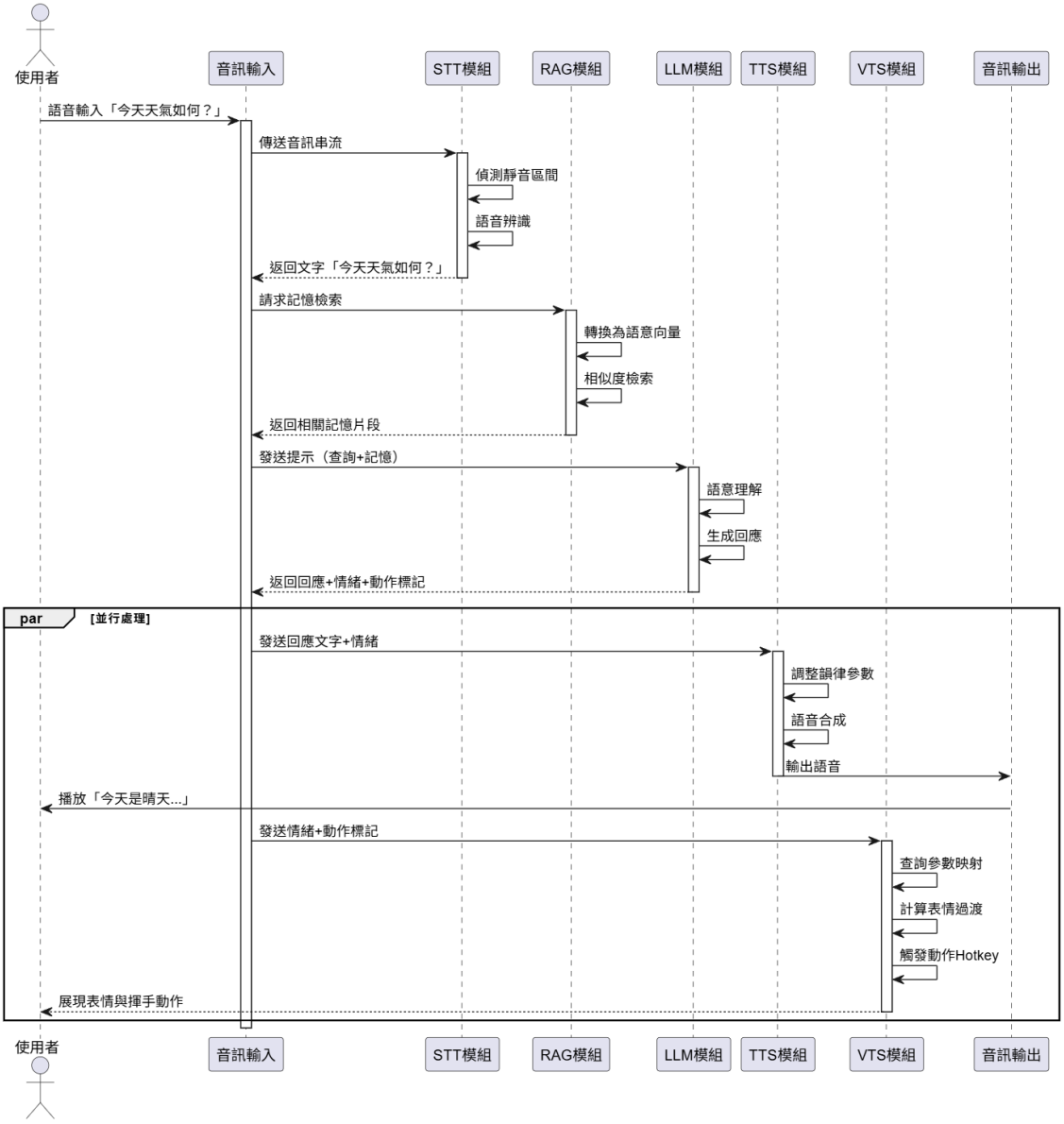

圖 3.2 語音對話互動循序圖

3.3 知識檢索問答

圖 3.3 為知識檢索問答之循序圖，展示系統如何透過語意檢索找出相關記憶片段，並結合檢索結果生成準確回應。使用者以文字方式詢問系統關於先前對話或預先載入之領域知識的問題，系統透過語意檢索找出相關記憶片段，結合檢索結果生成準確回應。此使用案例展示系統之記憶延續能力與知識回溯功能，使互動具備個性化與長期一致性。

3.3.1 前置條件

系統已完成啟動並建立記憶庫索引。記憶庫中存在一定數量之歷史對話記錄或領域知識內容。使用者透過支援之平台介面（如 Discord）輸入文字訊息。語言模型與檢索模組處於正常運作狀態。

3.3.2 主要流程

使用者輸入文字問題，如「我們上次討論的電影叫什麼名字？」。系統接收文字輸入後將其轉換為語意向量，並於記憶庫中執行相似度檢索。系統找出最相關之 3-5 個記憶片段，如「我們討論了《天能》這部電影，你覺得時間倒流的設定很有創意」。

系統將檢索結果與使用者問題一併組裝為提示，送入語言模型進行推理。語言模型根據檢索到之對話片段生成回應，如「我們上次討論的電影是《天能》，你對時間倒流的設定印象深刻呢！」。系統將回應內容顯示於平台介面，使用者確認答案正確並符合其記憶。

系統更新該記憶項目之存取時間與權重資訊，確保後續相關查詢能繼續找到此項目。當前對話內容亦加入會話記憶，維持多輪對話之上下文連貫性。若使用者繼續追問相關問題，系統能基於已檢索之記憶與當前對話脈絡提供更深入回應。

3.3.3 預期結果

系統能準確回溯先前對話內容或領域知識，回應與使用者期待相符。檢索結果與實際語意相關性高，不出現明顯不相關之記憶片段。回應內容自然流暢，能適當引用檢索結果而不顯生硬。使用者感受到系統「記得」先前互動，增強陪伴感與互動連續性。

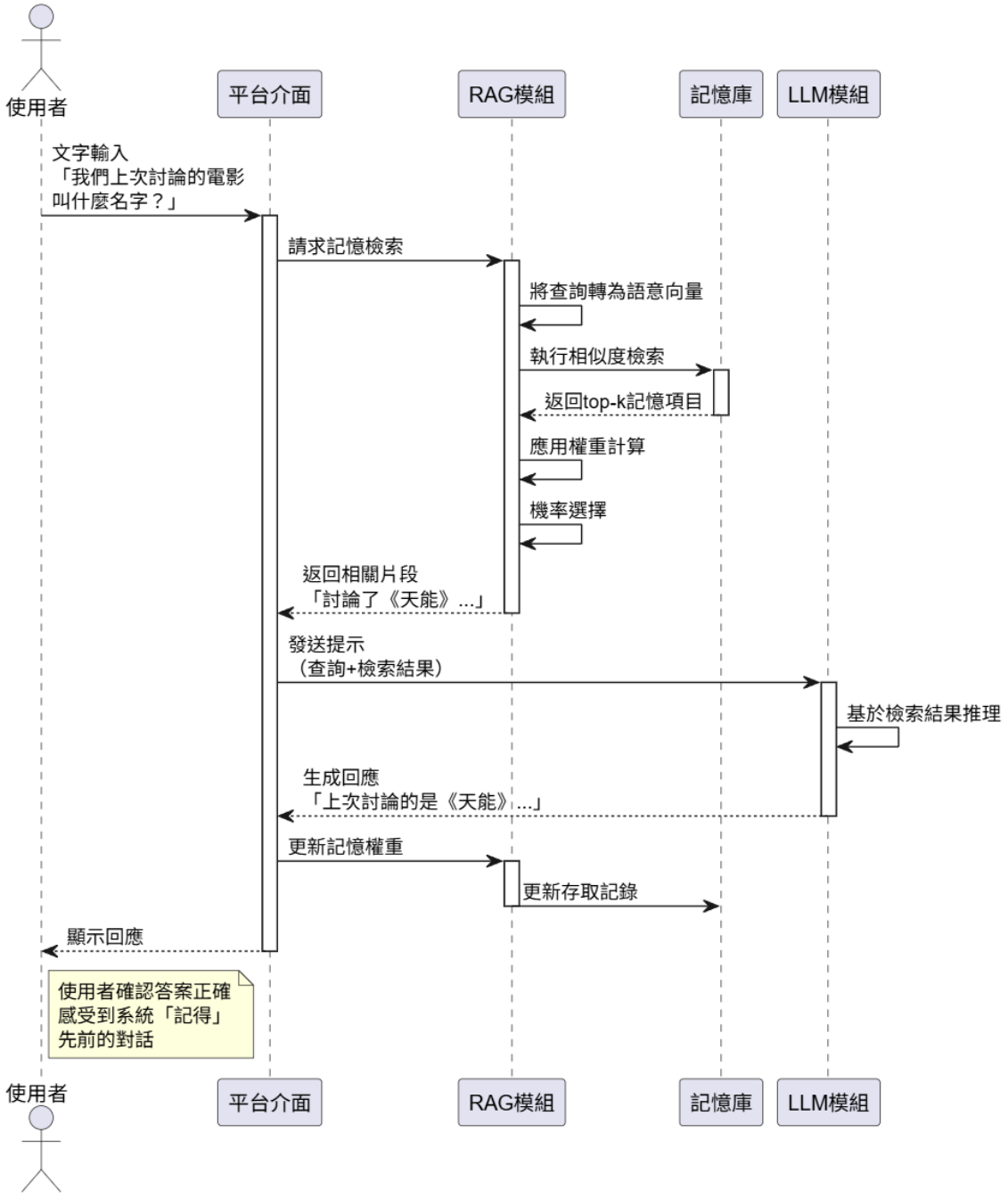

圖 3.3 知識檢索問答循序圖

3.4 情緒表達展示

圖 3.4 為情緒表達展示之循序圖，展示系統如何根據對話內容判斷情緒，並協調語音合成與虛擬角色控制模組實現一致之情感表達。使用者與系統對話過程中，系統根據對話內容自動判斷適當情緒，驅動虛擬角色展現對應表情與動作，實現語意與視覺之一致表達。此使用案例展示系統之情感理解與多模態表現能力，提升互動沉浸感。

3.4.1 前置條件

系統已完成啟動並連接至 VTube Studio。虛擬角色模型已正確載入，動畫參數映射表已初始化。Hotkey 綁定設定完成，系統能觸發預設動作與特效。語言模型支援結構化輸出，能生成包含情緒與動作標記之回應。

3.4.2 主要流程

使用者輸入或說出具有明顯情緒色彩之內容，如「太好了！我今天考試考了滿分！」。系統理解使用者表達之喜悅情緒，並生成祝賀性質之回應，如「哇！恭喜你考了滿分！你一定很努力準備呢！」，附帶情緒標記「快樂」與動作標記「拍手」。

系統將回應文字送入語音合成模組，根據「快樂」情緒調整語速加快、音調提高，生成充滿活力之語音輸出。同時，虛擬角色控制模組接收情緒與動作標記，查詢參數映射表找出「快樂」對應之臉部表情參數（如眼睛開合度增大、嘴角上揚、臉部略微傾斜），並啟動平滑過渡動畫。當表情過渡完成後，系統觸發「拍手」動作對應之 Hotkey，虛擬角色展現拍手特效。

使用者看到虛擬角色以愉悅表情回應並展現拍手動作，感受到系統理解其情緒並給予正向回饋。此互動過程強化使用者與虛擬角色之情感連結，提升互動體驗之真實感與參與感。

3.4.3 預期結果

虛擬角色表情變化與回應內容之情緒相符，不出現情緒不一致之情況（如悲傷內容配合快樂表情）。表情過渡平滑自然，無突兀之參數跳變。動作觸發時機適當，與語音內容同步或略有延遲但不影響體驗。使用者感受到系統具備情感理解能力，互動不僅停留於資訊交換層面。

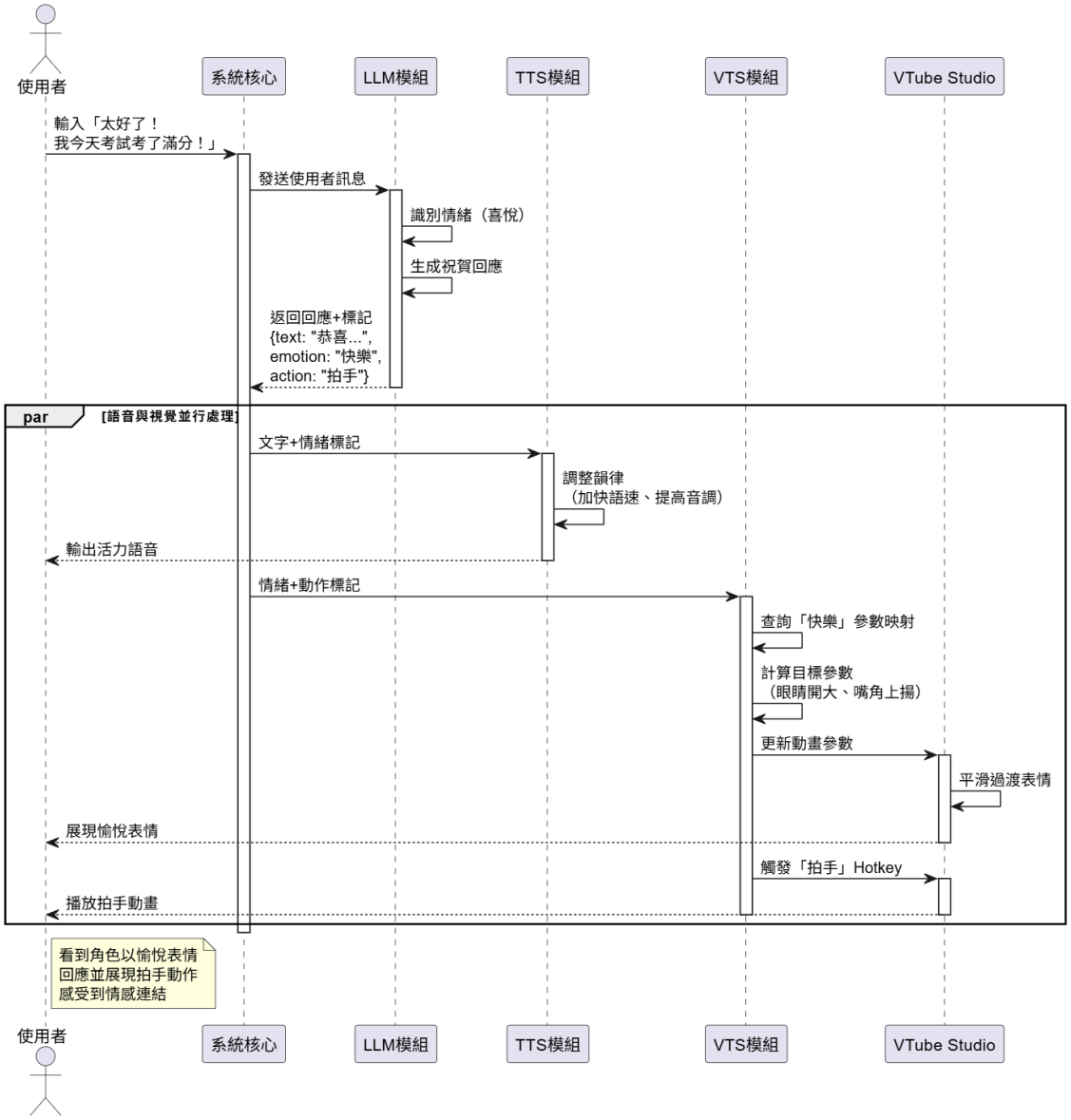

圖 3.4 情緒表達展示循序圖

3.5 跨平台訊息處理

圖 3.5 為跨平台訊息處理之循序圖，展示平台特定訊息如何透過介面模組轉換為統一格式，經核心邏輯處理後再轉回平台格式之完整流程。使用者透過不同通訊平台（如 Discord、Telegram）與系統互動，系統能正確接收與回應訊息，並適應各平台之訊息格式與功能特性。此使用案例展示系統之平台整合能力與部署彈性，使系統能快速接入既有通訊生態。

3.5.1 前置條件

系統已完成啟動並載入對應平台介面模組。平台機器人帳號已完成註冊與認證，擁有必要之訊息收發權限。系統配置檔案中已正確設定平台 Token 與連線參數。平台伺服器與系統網路連線正常。

3.5.2 主要流程

使用者於 Discord 伺服器中提及機器人並輸入訊息，如「@Norgo 推薦一部科幻電影」。Discord 平台介面模組接收到訊息事件，提取使用者識別碼、文字內容與頻道資訊，轉換為系統內部統一之訊息物件並發布至事件總線。

系統核心處理邏輯接收訊息物件，識別此為一般對話類型請求。系統執行記憶檢索、語言模型推理等標準流程，生成回應內容如「我推薦《銀翼殺手 2049》，它的視覺效果非常震撼，劇情也很有深度！」。

回應內容交由 Discord 平台介面模組進行格式轉換，組裝為 Discord 嵌入式訊息格式（Embed），包含回應文字、虛擬角色頭像與時間戳記。平台介面模組呼叫 Discord API 將訊息發送回原頻道，使用者於 Discord 中看到機器人之回覆。

當系統部署於多個平台時，相同之核心處理邏輯能同時服務來自不同平台之請求。平台特定之功能差異（如 Discord 支援語音頻道、Telegram 支援內聯鍵盤）透過平台介面模組封裝處理，核心邏輯不需關注平台細節。

3.5.3 預期結果

系統能正確接收與解析不同平台之訊息，不因平台格式差異導致處理失敗。回應內容能適當轉換為平台特定格式並正確發送。當平台支援特定功能時(如嵌入式訊息、

快速回覆按鈕)，系統能充分利用以提升互動體驗。新增平台支援時僅需實作平台介面模組，不影響核心處理邏輯與既有平台功能。

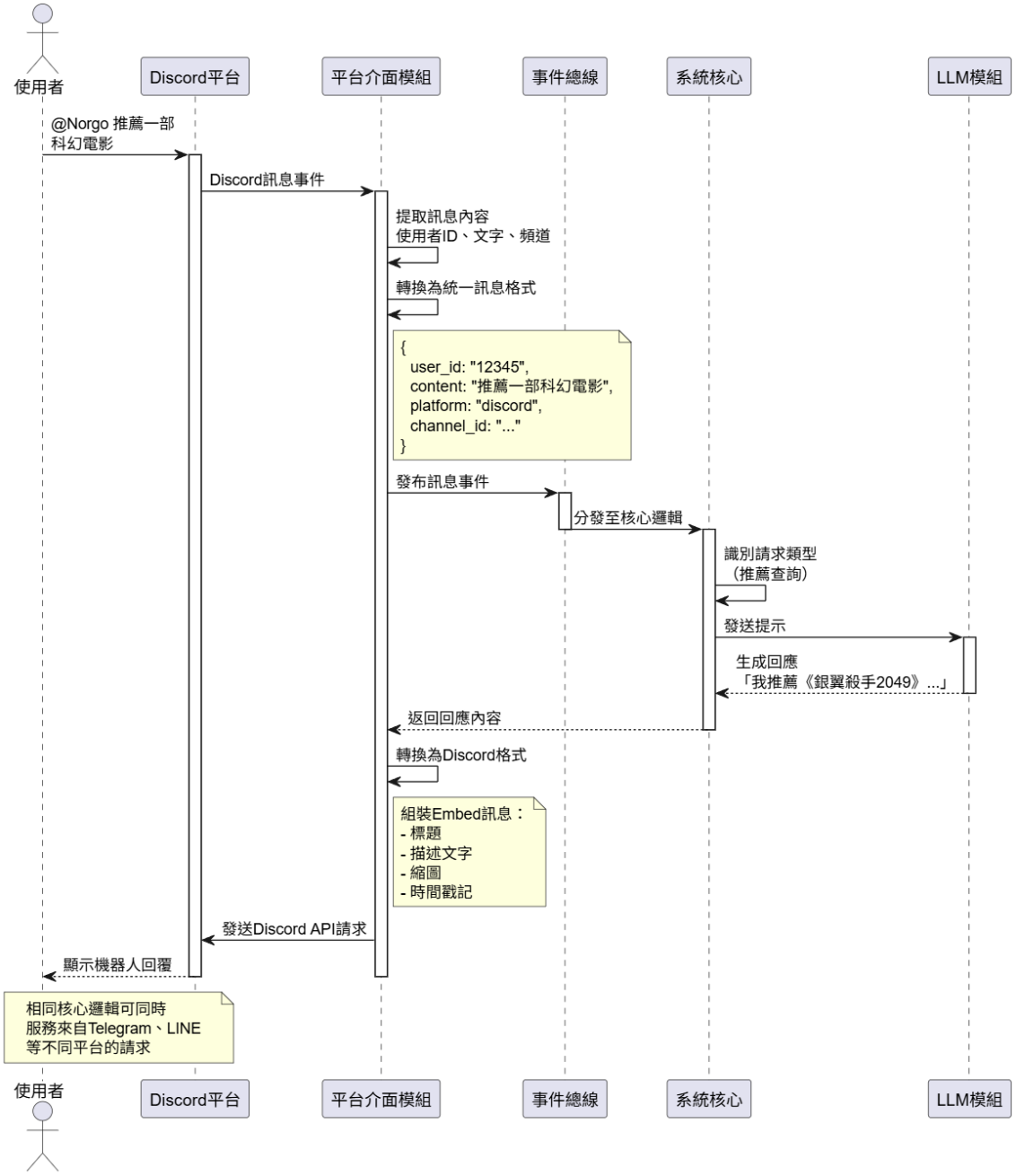

圖 3.5 跨平台訊息處理循序圖

系統設計規格書

目錄

目錄 i
表目錄 ii
圖目錄 iii
第 1 章 設計概述 1
1.1 設計目標與原則 1
1.2 設計限制與假設 1
第 2 章 系統架構設計 3
2.1 整體系統架構 3
2.2 架構模式與設計模式 5
2.3 模組劃分與職責 6
2.4 部署架構 7
2.5 語音辨識模組（STT） 8
2.6 語言模型推理模組（LLM） 9
2.7 檢索增強模組（RAG） 9
2.8 語音合成模組（TTS） 11
2.9 虛擬角色控制模組（VTS） 12
第 3 章 資料設計 14
3.1 資料模型 14
3.2 向量資料庫設計 15
3.3 記憶模組資料結構 16
3.4 資料流設計 17
第 4 章 介面設計 18
4.1 模組間資料傳遞結構 18
4.2 模組介面契約 19
4.3 事件驅動機制 20
參考文獻 22

表目錄

表 2.1 核心模組職責對照表 7
表 4.1 Platform Message 欄位定義 18
表 4.2 Transcript Result 欄位定義 19
表 4.3 Memory Item 欄位定義 19
表 4.4 LLM Response 欄位定義 19
表 4.5 系統事件類型定義 21

圖目錄

圖 2.1 Norgo 系統整體架構圖 4
圖 2.2 事件驅動架構示意圖 6
圖 2.3 VTS 模組內部流程圖.....13

第1章 設計概述

本章建立 Norgo 專案設計規格書之基礎框架，說明設計指導原則與限制條件。本文件作為連結需求規格與實作細節之關鍵文件，為後續架構設計、模組設計與演算法設計提供一致性之指導方針。

1.1 設計目標與原則

本專案為一基於 Python 開發之 AI Agent 互動系統，整合語音辨識（Speech-to-Text, STT）、大型語言模型（Large Language Model, LLM）、檢索增強生成（Retrieval-Augmented Generation, RAG）、語音合成（Text-to-Speech, TTS）與虛擬主播動畫控制（VTube Studio, VTS）等核心技術，實現多模態人機互動能力。本專案之功能需求與效能指標已於系統需求規格書中定義，開發方法論與時程規劃已於專案執行規劃書中說明，本節聚焦於技術實作層面之設計原則。

本專案遵循六項核心設計原則。首先，採用低耦合（Low Coupling）與高內聚（High Cohesion）之架構策略，各模組間透過統一介面協議通訊，模組內部實作細節對外部完全隱藏，確保單一模組變更不影響整體運作。其次，採納事件驅動（Event-Driven）設計，透過非同步事件監聽機制與裝飾器模式註冊事件處理函式，支援高併發任務處理並提升資源使用效率。第三，運用工廠模式（Factory Pattern）統一管理實例化流程，開發者僅需提供配置參數即可初始化所需模組，簡化初始化流程並支援模組動態切換。第四，遵循開放封閉原則（Open-Closed Principle），新增功能應透過新增模組或擴展現有介面實現而非修改核心程式碼，確保穩定性與向後相容性。第五，以語意優先（Semantics-First）為核心思維，所有技術選型與演算法設計皆以提升語意準確度與回應適切性為首要考量。第六，採用非同步處理架構，基於 Python asyncio 框架實現併發任務管理，確保耗時操作以非阻塞方式執行。

1.2 設計限制與假設

本專案之設計與實作過程受到多項技術因素之制約，同時建立於若干基礎假設之上。本節說明這些限制因素如何影響架構設計決策。

在技術限制方面，本專案需於一般消費級硬體環境下運行，GPU 記憶體容量限制使專案無法部署超大型語言模型，團隊因此採用混合部署策略，結合輕量級本地模型與雲端 API 服務。處理器運算能力限制亦影響向量檢索演算法之複雜度，專案採用

FAISS 近似最近鄰搜尋技術以平衡檢索準確度與速度要求。語音互動場景對延遲具有嚴格要求，此即時性限制影響語音辨識模組需採用快速批次處理策略、RAG 檢索流程捨棄計算密集之 reranking 階段、語言模型推理需限制生成長度，以及表情動畫控制需預先載入參數映射表以降低查詢延遲。具體之硬體需求規格已於系統需求規格書中定義，資源限制對研究範疇之影響已於專案執行規劃書中說明。

在虛擬角色控制方面，Live2D 模型之參數結構與反應特性存在個體差異。不同模型對相同動畫參數之反應範圍與靈敏度可能顯著不同，導致相同情緒指令產生非預期之視覺效果。本專案透過參數正規化與範圍限制機制降低此問題影響，但完整之跨模型一致性控制需要存取 Cubism SDK 之模型描述格式，此功能列入未來改進方向。

本專案設計建立於以下基礎假設之上：使用者具備基本語音輸入設備與網路連線能力，音訊品質符合語音辨識模組之最低要求；部署環境具備 Python 執行環境與必要相依套件，並能存取所需之 API 服務端點；VTube Studio 軟體已正確安裝並啟用 API 功能，虛擬角色模型已載入且 Hotkey 設定已完成；語言模型能理解並遵循設定之提示格式，生成符合預期結構之回應內容；向量嵌入模型能有效捕捉中文語意特徵，檢索結果與實際語意相關性具備正相關性。

第2章 系統架構設計

本章說明 Norgo 專案之整體架構設計、模組組織方式與技術實作策略。架構設計為將需求規格轉化為可實作技術方案之關鍵階段，需明確定義各模組職責、介面規範與協作機制，確保專案具備可維護性、可擴展性與可測試性。

2.1 整體系統架構

Norgo 專案採用分層式架構設計，將語音互動流程分解為輸入層、處理層與輸出層三個主要層級。輸入層負責接收使用者之文字或語音訊息，處理層執行語意理解、知識檢索與推理生成，輸出層將回應轉換為文字或語音形式並驅動虛擬角色動畫。各層間透過中央控制模組協調，形成完整之互動閉環。專案整體架構如圖 2.1 所示。

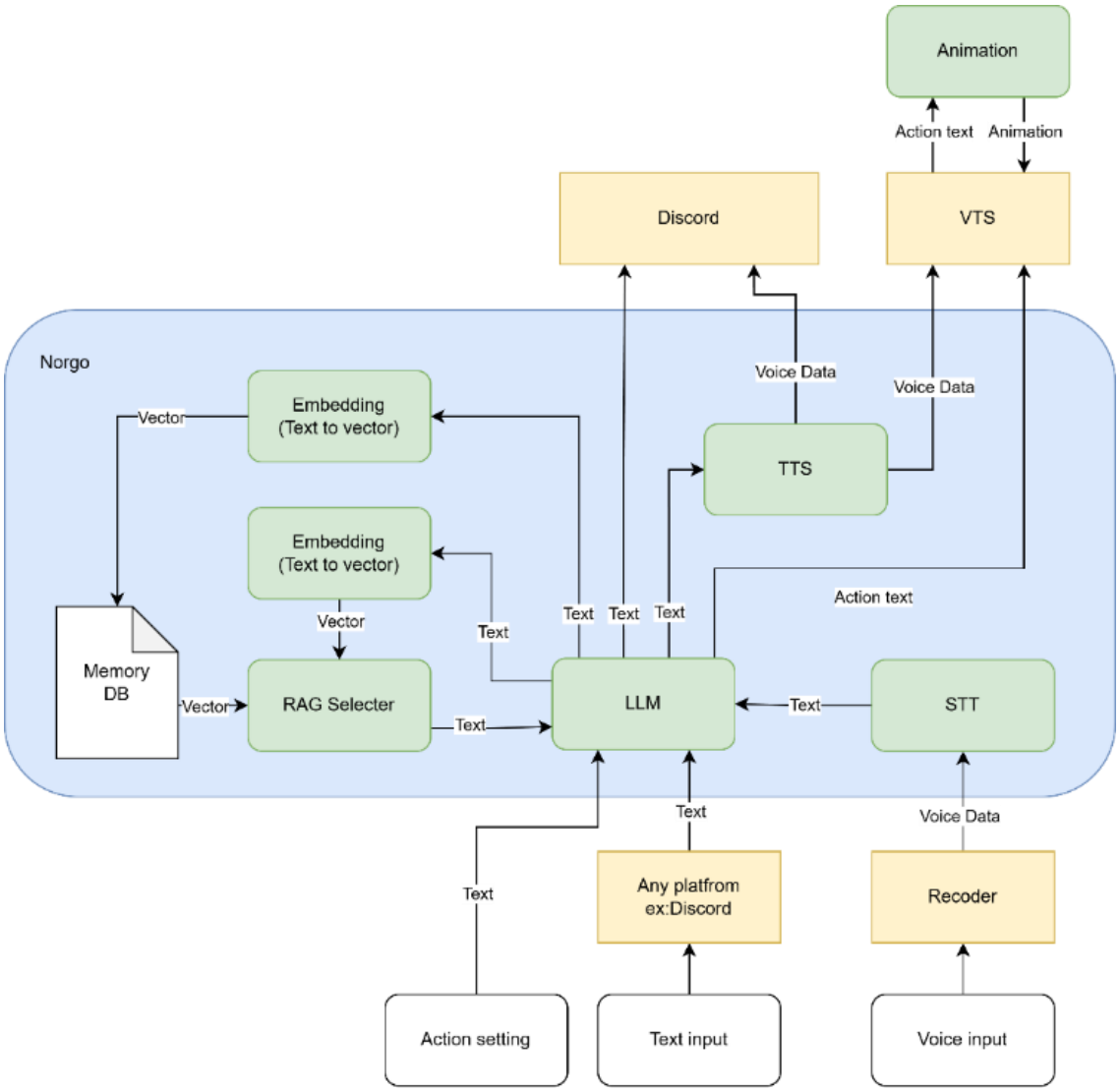

圖 2.1 Norgo 系統整體架構圖

於文字輸入情境下，使用者透過平台介面輸入問題，訊息首先由中央控制模組進行預處理並轉換為語意向量。專案透過 RAG 模組於語意知識資料庫中檢索相關段落，將檢索結果與使用者問題一併傳遞至 LLM 模組進行推理。LLM 模組生成回應內容後，經由中央控制模組路由至輸出介面，完成一次互動循環。於語音互動情境中，專案流程擴展為更完整之端到端處理鏈：使用者語音訊號經由麥克風輸入，STT 模組（採用 Whisper 模型）將語音轉換為文字，後續處理流程與文字輸入情境相同；LLM 生成回應後，TTS 模組將文字內容合成為語音輸出，同時 VTS 模組根據回應之情緒與

動作指令驅動 Live2D 虛擬角色展現對應表情與動作。此多模態輸出機制使專案能提供視聽覺結合之沉浸式互動體驗。

架構設計之關鍵特徵在於中央控制模組之協調角色。中央控制模組不直接實作任何業務邏輯，而是作為訊息路由與事件分發中心，管理各模組之生命週期並協調模組間之資料傳遞。此設計使各功能模組能保持獨立性，模組之新增、替換或移除不影響其他模組運作。例如，專案可同時支援 Discord、Telegram 與 LINE 等多種平台介面，各平台模組僅需實作統一之訊息介面即可接入，平台切換不需修改核心處理邏輯。專案採用非同步處理架構，基於 Python asyncio 框架實現併發任務管理，語音辨識、語言模型推理、向量檢索與語音合成等耗時操作均以非阻塞方式執行，確保能同時處理多個使用者請求而不產生相互阻塞。架構亦預留擴展空間以支援未來功能整合，例如視覺理解模組介面已於架構中定義，待後續整合圖像辨識能力時，僅需實作對應介面並註冊至中央控制模組即可啟用。

2.2 架構模式與設計模式

本專案於架構層級採用事件驅動架構（Event-Driven Architecture），並於實作層級運用工廠模式（Factory Pattern）、裝飾器模式（Decorator Pattern）等經典設計模式，構建鬆散耦合且易於擴展之系統結構。

專案核心採用事件驅動架構，基於 Python asyncio 框架實現非同步事件循環。各模組透過事件發布與訂閱機制進行通訊，當特定事件觸發時，所有註冊該事件之處理函式將被非阻塞式呼叫。此架構使專案能以反應式方式處理使用者輸入、模型推理完成、語音合成結束等各類事件，避免模組間直接依賴。例如，當 STT 模組完成語音辨識後，發布 on_speech_recognized 事件並附帶辨識結果，訂閱此事件之 RAG 模組與 LLM 模組隨即啟動處理流程，無需 STT 模組知曉後續處理邏輯。事件驅動架構於專案中之具體實現採用裝飾器模式進行事件註冊，開發者透過 Python 裝飾器語法標註事件處理函式，專案於初始化階段自動掃描並註冊這些處理函式至事件分發器，使事件處理邏輯之新增與移除變得簡潔直觀。事件驅動架構示意圖如圖 2.2 所示。

專案採用工廠模式管理各模組之實例化過程。各核心模組（STT、LLM、RAG、TTS、VTS）均定義抽象基礎類別，具體實作類別透過工廠類別進行建構。工廠類別根據配置檔案中指定之模型類型與參數，自動選擇對應實作類別並完成初始化，使模組替換過程高度簡化，開發者僅需修改配置檔案即可切換不同模型或服務提供者。以 LLM 模組為例，工廠類別支援 OpenAI API 模型（如 GPT-4o-mini）、本地開源模型

（如 LLaMA 3.2-3B Instruct）與蒸餾模型（如 DeepSeek-R1-Distill-Qwen-1.5B）之動態建構。工廠模式之實作透過策略模式進行增強，各模型實作類別均實現統一之推理介面，確保工廠產生之實例可互換使用而不影響呼叫方程式碼。於資料處理流程方面，專案運用管道模式（Pipeline Pattern）組織多階段處理邏輯，使用者輸入經過預處理、向量化、檢索、推理、後處理等階段，各階段作為獨立處理單元串接為處理管道，使各處理階段可獨立開發與測試，並支援動態調整管道結構以適應不同應用場景。

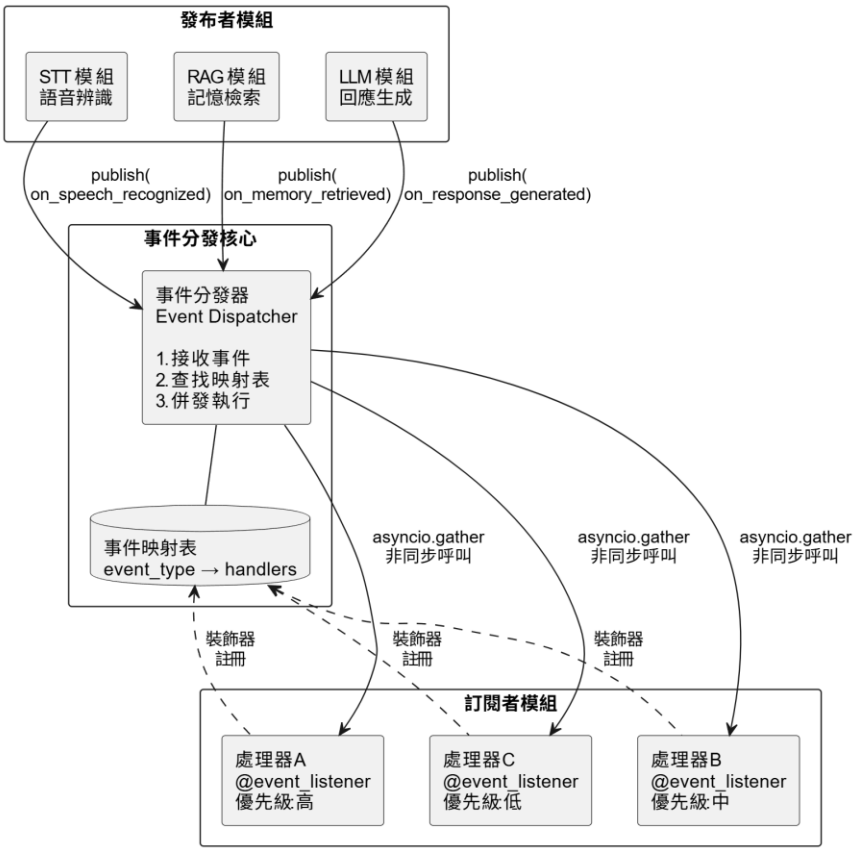

圖 2.2 事件驅動架構示意圖

2.3 模組劃分與職責

NNorgo 專案依據功能職責劃分為七個核心模組，各模組負責特定處理階段並透過統一介面協作。各模組之詳細內部設計、演算法實作與 API 規範將於第 3 章完整呈現。

中央控制模組（Core Controller）負責專案整體流程協調與模組生命週期管理，作為訊息路由中心接收外部輸入並分發至對應處理模組，同時管理事件註冊、模組初始化與配置載入，維護對話上下文並追蹤多輪對話之歷史記錄。語音辨識模組（STT Module）負責將使用者語音輸入轉換為文字，封裝 Whisper 模型之推理流程，支援即時串流處理與批次處理模式，並執行音訊預處理包括降噪、音量正規化與取樣率轉換。語言模型模組（LLM Module）負責語意理解與回應生成，透過工廠模式支援多種語言模型後端，包括 OpenAI API、本地開源模型與蒸餾模型，支援純文字模式與 JSON 結構化模式輸出。檢索增強模組（RAG Module）負責語意記憶之儲存與檢索，整合語意嵌入模型（jina-embeddings-v3）與向量資料庫（FAISS）實現高效之相似度檢索，核心功能包含記憶寫入、語意檢索與記憶權重更新。語音合成模組（TTS Module）負責將文字內容轉換為語音輸出，透過工廠模式支援 BreezyVoice、XTTS-v2 與 Coqui TTS 等多種後端，並提供串流輸出選項以降低首字延遲。虛擬角色控制模組（VTS Module）負責驅動 Live2D 虛擬角色展現表情與動作，透過 VTube Studio API 控制動畫參數，維護情緒至動畫參數之映射表並實作表情過渡控制機制。平台介面模組（Platform Interface Module）負責與外部訊息平台整合，抽象化不同平台之訊息格式與 API 呼叫方式，目前已實作 Discord 平台介面。各模組職責對照如表 2.1 所示。

模組名稱	主要職責	輸入	輸出
中央控制模組	流程協調、事件管理	使用者訊息	分發至各模組
語音辨識模組	語音轉文字、音訊預處理	音訊資料	辨識文字
語言模型模組	理解語意、生成回應	訊息內容	回應文字
檢索增強模組	記憶儲存與檢索	查詢文字	相關記憶
語音合成模組	文字轉語音、情緒表達	文字內容	音訊資料
虛擬角色控制模組	驅動角色動畫、表情控制	情緒標籤 動作指令	動畫參數
平台介面模組	平台整合、訊息格式轉換	平台訊息	標準化訊息

表 2.1 核心模組職責對照表

2.4 部署架構

Norgo 專案採用單機部署架構，所有核心模組於同一運算環境中執行，透過程序內通訊進行協作。此部署策略簡化配置與維護流程，降低分散式架構帶來之網路延遲與通訊複雜度，適合中小規模應用場景與原型驗證需求。

專案執行環境建立於 Python 3.10 以上版本，依賴 asyncio 事件循環作為並發處理基礎。所有模組於單一 Python 程序內以協程形式執行，透過事件循環排程實現非阻塞式並發處理，使專案能於有限硬體資源下同時處理多個使用者請求，避免多程序或多執行緒架構帶來之資源開銷與同步複雜度。核心模組依據運算特性可分為三種部署類型：本地推理模組包括 STT 模組、本地 LLM 模組與 RAG 模組之向量嵌入運算，此類模組直接於部署主機上執行模型推理，需消耗 CPU 與 GPU 資源，Whisper 模型支援 GPU 加速，專案優先使用 GPU 以降低延遲，當 GPU 不可用時自動退回 CPU 模式；遠端 API 模組包括雲端 LLM 模組與部分 TTS 服務，此類模組透過 HTTP 請求呼叫外部服務，本地僅需處理請求組裝與回應解析，API 呼叫採用非同步客戶端實作，支援請求逾時控制與自動重試機制；外部整合模組包括 VTS 模組與平台介面模組，此類模組透過 WebSocket 或平台 SDK 與外部應用程式通訊，VTS 模組要求 VTube Studio 軟體運行於同一主機或區域網路內，專案啟動時自動探測服務可用性，若連線失敗則禁用虛擬角色控制功能但不影響其他模組運作。

配置管理採用分層式策略，預設配置參數寫入程式碼常數，環境特定配置透過 YAML 或 JSON 檔案覆寫，敏感資訊如 API 金鑰透過環境變數注入。此設計使專案能於不同環境間快速部署而無需修改程式碼，並降低敏感資訊洩露風險。日誌輸出採用分級記錄機制，支援 DEBUG、INFO、WARNING、ERROR 四個級別，日誌內容包含時間戳記、模組名稱、事件類型與詳細訊息，輸出至標準輸出與檔案雙重目標，便於運作監控與問題診斷。部署架構已預留未來擴展空間，雖然目前採用單機部署，但各模組間透過訊息介面通訊而非直接函式呼叫，此設計使模組未來可獨立部署為微服務，透過訊息佇列或 RPC 機制進行跨程序通訊。

2.5 語音辨識模組（STT）

語音辨識模組負責將使用者語音輸入轉換為文字，為系統語音互動能力之基礎。模組採用三層架構設計：音訊輸入層接收麥克風或檔案之原始語音資料，預處理層執行音量正規化、降噪與取樣率轉換（統一至 16kHz），模型推理層封裝 Whisper 模型執行語音轉文字。模組支援串流與批次兩種處理模式，串流模式持續監聽音訊輸入並於偵測靜音區間時自動切分片段提交推理，批次模式一次性處理完整音訊檔案，兩種模式透過統一介面提供。

系統採用 OpenAI 開源之 Whisper 模型作為辨識核心 [2]。Whisper 為基於 Transformer 架構 [13] 之編碼器-解碼器模型，於超過 68 萬小時多語言語音資料上進行弱監督訓練，具備優異之多語系辨識能力與背景噪音容忍度。模型提供 tiny、base、

small、medium、large 等規模版本，系統預設採用 base 或 small 版本以平衡效能與準確度。模型整合透過官方 Python 套件實現，系統啟動時自動偵測 CUDA 可用性並優先使用 GPU 加速，當 GPU 不可用時自動退回 CPU 模式。

模型檔案於首次執行時自動下載並快取於本地。Whisper 支援自動語言偵測或由使用者指定目標語言，系統預設啟用自動偵測以適應多語言混合場景。模型輸出包含辨識文字、各片段時間戳記與信心分數，供後續模組進行可靠度評估與字幕生成。模組實作遵循工廠模式，透過 STTFactory 類別根據配置參數建構具體實例，介面設計已預留擴展空間供未來整合其他辨識引擎。具體之方法簽名與參數規範詳見第 4 章。

2.6 語言模型推理模組（LLM）

語言模型推理模組負責語意理解與回應生成，為系統自然語言處理之核心。模組採用分層設計：提示管理層整合系統提示、使用者訊息、對話歷史與檢索上下文，並依據不同模型格式進行適配；模型抽象層定義統一推理介面，封裝不同模型後端之呼叫細節；回應處理層解析模型輸出，擷取文字內容與結構化資訊並進行格式驗證。模組支援純文字與 JSON 結構化兩種輸出模式，後者用於需解析情緒標籤與動作指令之場景，系統透過提示工程與輸出格式約束確保模型遵循預期結構。

系統支援三類語言模型後端：雲端 API 模型、本地開源模型與蒸餾模型。考量效能與資源限制，系統選定 GPT-4o-mini [11] 作為主力推理引擎，此模型具備穩定反應速度、原生 JSON Mode 支援與良好語言能力；同時保留 LLaMA 3.2-3B Instruct [12] 作為本地備援方案，適用於離線部署場景。模組透過工廠模式管理多模型支援，LLMFactory 類別根據配置檔案指定之模型類型動態建構對應實作，各實作類別均繼承 BaseLLM 抽象基礎類別並實作統一之 generate 方法，確保不同模型可互換使用。模型切換僅需修改配置檔案，無需變更程式碼。具體之 API 呼叫格式與參數規範詳見第 4 章。

2.7 檢索增強模組（RAG）

檢索增強模組負責語意記憶之儲存與檢索，為系統提供對話上下文延續能力。模組採用三層架構：向量嵌入層將文字內容轉換為固定長度之語意向量，檢索層基於向量資料庫執行高效近似最近鄰搜尋，記憶管理層負責記憶項之新增、更新與刪除並維護權重資訊。模組維護兩類資料結構：語意向量索引儲存對話片段之向量表示以支援

快速相似度檢索，記憶權重表儲存各記憶項之歷史權重值用於機率選擇與偏好調整，兩者透過記憶項識別碼關聯。

系統採用 JinaAI 之 jina-embeddings-v3 作為主要語意向量模型 [8]，此模型為開源架構且於多項測試中展現良好穩定性與語意捕捉能力，同時支援 OpenAI text-embedding-3-small 作為備援選項。向量檢索採用 Facebook FAISS 向量資料庫 [3, 4]，透過產品量化與倒排索引等技術支援億級資料規模下之高效搜尋，系統採用 IndexFlatIP 記憶體索引結構以降低檢索延遲，資料持久化透過定期匯出向量與元資料至磁碟實現。檢索流程參考 Lewis 等人提出之 RAG 架構概念 [1]，但針對即時互動場景捨棄計算密集之 cross-encoder reranking 階段，改以輕量化機率加權機制進行候選項選擇。

系統設計動態權重更新機制，綜合考量語意相關性與歷史引用頻率以避免特定高頻項持續壟斷回應。每次查詢後根據語意相似度與歷史記錄更新權重，公式如下：

$$w_i = \beta \cdot w_i + \alpha \cdot (M - s_i) \tag{1}$$

其中：

- w_i為資料項 i 的歷史權重；
- s_i為其與當前輸入語句之語意相似度；
- M為目前候選集中相似度的中位數；
- β為時間衰減因子（decay factor）；
- α為學習速率（learning rate）。

該設計使得歷史上與輸入語句表現偏離中位數的資料項將獲得額外調整，進一步調節權重分佈並避免特定高頻項持續壟斷回應。

在樣本選擇階段，系統會根據以下公式綜合計算每筆候選資料的回應得分：

$$score_{\mathrm{i}} = w_{\mathrm{i}}^{0.3} \times \frac{1}{(s_{\mathrm{i}} + \epsilon)} \tag{2}$$

其中 ϵ 為避免分母為零之穩定係數。此公式設計意圖如下：

- 使用歷史權重 w_i 表示偏好強度，並透過指數 0.3 進行平滑處理；
- 相似度反比項則強化輸入與資料項之語意接近程度；

- 權重與語意分數交叉組合後可產生兼顧「相關性」與「偏好性」之選擇標準。

隨後，系統透過 Softmax 函數將所有分數正規化為機率分佈：

$$p_i = \frac{e^{\frac{score_i}{T}}}{\sum_j e^{\frac{score_j}{T}}} \quad (3)$$

其中，溫度參數 T 可調節分佈之離散程度，提供一種探索（exploration）與利用（exploitation）間的權衡機制。當 T 趨近 0 時，系統更傾向選擇單一高分項；反之則促進機率分散以提升回應多樣性。此機制已整合至檢索流程，初步實驗顯示能有效避免重複回應並提升內容適切性。具體之方法簽名與參數規範詳見第 4 章。

2.8 語音合成模組（TTS）

語音合成模組負責將文字內容轉換為語音輸出，為系統提供語音回應能力。模組採用三層架構：文字預處理層執行文字正規化、數字轉換與標點符號處理，模型推理層封裝不同 TTS 模型之合成流程，音訊後處理層對生成音訊進行音量調整與格式轉換。模組支援同步與非同步兩種合成模式，並提供串流輸出選項以逐片段生成音訊降低首字延遲。

系統預設採用 BreezyVoice 作為主要語音合成引擎。BreezyVoice 為聯發科技開源之台灣普通話 TTS 模型 [5]，基於 CosyVoice 架構整合 S^3 tokenizer、LLM 與最優傳輸條件流匹配模型（OT-CFM），專為台灣腔調進行聲學特性優化。該模型僅需五秒語音樣本即可完成高自然度語音克隆，並支援語速、語調與情緒調整，於語音自然度與音素準確率方面優於現有商用系統，在一般語境與中英混合場景中均展現優異表現 [5]。

本專案針對 BreezyVoice 進行兩項關鍵修改以適應系統需求。首先，原始 BreezyVoice 於每次合成時皆需重新計算說話人嵌入向量，此過程涉及參考音訊之編碼與特徵提取，對即時互動場景造成顯著延遲。專案實作說話人嵌入快取機制，於首次載入參考音訊時計算嵌入向量並序列化儲存至本地檔案，後續合成直接載入快取向量，將說話人初始化時間降低。其次，原始 BreezyVoice 僅支援 Linux 平台，部分相依套件與音訊處理邏輯存在平台限制。專案重構相關程式碼，替換平台相依之音訊後端並調整檔案路徑處理邏輯，使模組可於 Windows 與 macOS 環境下正常運作，提升開發與部署彈性。

系統同時整合 XTTS-v2 作為備援方案 [6, 7]。XTTS-v2 為 Coqui 推出之多語言語音合成模型，整合 VQ-VAE 編碼器、GPT-2 解碼器與 HiFi-GAN 語音生成器 [9, 10]，支援 17 種語言之 zero-shot 語音克隆，採用 Perceiver 模型進行說話人資訊條件化，允許使用多個參考音訊捕捉目標說話人不同面向特徵。當系統需要非中文語言輸出或 BreezyVoice 不可用時，自動切換至 XTTS-v2。模型切換透過工廠模式管理，TTSFactory 類別根據配置檔案動態建構對應實作。

模組提供統一之參數控制介面，核心參數包括語速（0.5-2.0）、音調（-1.0-1.0）、音量（0.0-1.0）與情緒強度（0.0-1.0），各參數透過正規化數值範圍確保不同模型後端之參數語意一致。系統支援情緒類型參數，根據 LLM 輸出之情緒標籤自動選擇對應韻律模式，目前支援中性、快樂、悲傷、驚訝、生氣與害羞六種基本情緒，各情緒預定義語速、音調與韻律變化參數組合。參數控制採用策略模式實作，各 TTS 模型實作類別負責將統一參數映射至模型特定格式。具體之方法簽名與參數規範詳見第 4 章。

2.9 虛擬角色控制模組（VTS）

虛擬角色控制模組負責驅動 Live2D 虛擬角色展現表情與動作，為系統提供視覺互動能力。模組採用分層架構：API 通訊層負責與 VTube Studio 建立 WebSocket 連線並處理訊息收發，情緒處理層解析 LLM 輸出之情緒標籤與動作指令並映射至對應動畫參數，動畫控制層負責參數數值計算與變化曲線生成以實現平滑過渡效果，參數管理層維護動畫參數狀態並追蹤當前值與目標值。模組內部流程可區分為初始化、事件監聽與動畫執行三個階段，如圖 2.3 所示。模組採用事件驅動設計，透過訂閱 LLM 模組發布之情緒與動作事件觸發動畫控制流程，使模組與其他元件保持解耦。

系統透過 VTube Studio 提供之 API 插件功能與虛擬角色應用程式通訊。VTS API 採用 WebSocket 協議支援即時雙向通訊，延遲可控制於毫秒等級。系統於啟動時透過 API 進行認證並建立持久連線，支援三類核心操作：參數讀取操作獲取當前 Live2D 模型之可用參數列表與數值範圍，參數設定操作修改指定參數之目標數值，Hotkey 觸發操作啟動預先綁定之特效與動作用於實現揮手、冒汗等瞬時效果。系統於初始化階段自動掃描並註冊所有可用 Hotkey，建立動作指令與 Hotkey 識別碼之映射表，新增動作僅需於 VTS 中設定 Hotkey 並更新映射表即可。模組實作包含連線重試與錯誤處理機制，當連線中斷時自動嘗試重新連線，當目標模型不支援特定參數時記錄警告並跳過該參數設定，避免系統異常中止。

表情動畫參數設計為虛擬角色情緒表達之核心，系統定義三大類動畫參數。臉部轉動控制包括左右轉動、上下轉動與傾斜三個自由度，對應 Live2D 之 FaceAngleX、FaceAngleY 與 FaceAngleZ 參數，不同情緒對應不同轉動範圍，如快樂情緒設定較大傾斜角度表現活潑感，悲傷情緒略微下傾表現低落感。眼口控制參數包括眼睛開合、眼球移動與嘴型變化，系統根據情緒狀態調整開合度並透過 Perlin noise 函數生成自然視線漂移效果，避免僵硬凝視狀態。眉毛與表情線條控制包括眉毛高度、角度與臉部裝飾元素如臉頰紅暈與汗滴之顯示強度。各參數於初始化時依據情緒配置基本值，並於運行時疊加基於 Perlin noise 之微變動，模擬實際人物之微表情特徵以提升視覺真實感。

系統設計過渡動畫機制處理參數於不同情緒狀態間之變化。過渡控制基於線性插值方法計算當前值與目標值間之中間狀態，並引入 easing 函數進行時間軸加權處理，模擬動畫啟動與收尾時之緩入緩出效果。系統目前支援線性、二次緩入緩出與三次緩入緩出三種 easing 函數，線性插值適用於中性情緒過渡，二次曲線適用於溫和情緒變化，三次曲線適用於劇烈情緒變化如驚訝至生氣。情緒與 easing 函數之對應關係透過配置表預先定義，當情緒切換涉及多參數調整時，系統確保所有參數以相同時長與曲線執行過渡以維持視覺協調性。值得注意的是，不同 Live2D 模型對相同參數之反應存在差異，系統透過參數正規化與範圍限制機制降低此問題影響，完整之跨模型一致性控制列入未來改進方向。具體之方法簽名與參數規範詳見第 4 章。

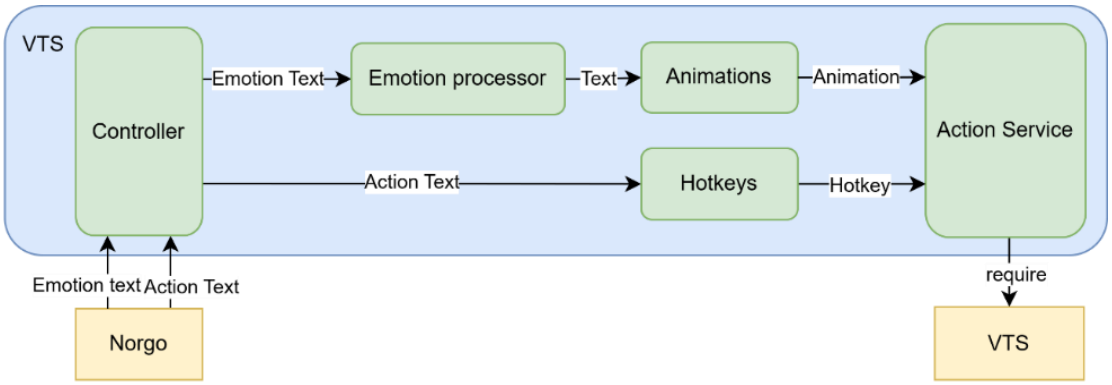

圖 2.3 VTS 模組內部流程圖

第3章 資料設計

本章說明 Norgo 系統之資料模型、資料結構與資料流設計。資料設計為系統實作之重要基礎，定義資訊如何儲存、組織與傳遞，直接影響系統效能、可維護性與擴展性。

3.1 資料模型

系統資料模型定義核心業務實體及其關聯關係，確保資料一致性與完整性。本節說明訊息、記憶、情緒狀態與動畫參數四類核心實體之結構設計。

訊息實體表示系統與使用者間之單次通訊內容，包含訊息識別碼、角色標籤、內容文字、時間戳記與平台資訊等屬性。角色標籤區分訊息來源為使用者或系統，用於對話歷史管理與提示組裝。內容文字儲存實際訊息內容，支援純文字與結構化 JSON 兩種格式。時間戳記記錄訊息產生時間，用於對話排序與時效性判斷。平台資訊記錄訊息來源平台如 Discord 或 Telegram，用於平台特定邏輯處理。

記憶實體表示系統記憶庫中之單筆記憶項，包含記憶識別碼、內容文字、語意向量、時間戳記、權重值與元資料等屬性。記憶識別碼為唯一識別符，用於關聯向量索引與權重表。內容文字儲存記憶之原始文字內容，便於回溯與展示。語意向量為嵌入模型產生之固定長度向量表示，用於相似度計算。權重值記錄記憶項之歷史選擇權重，用於機率選擇與多樣性控制。元資料包含記憶類型、來源對話識別碼與使用者標籤等輔助資訊。

情緒狀態實體表示系統當前或目標情緒狀態，包含情緒類型、強度值、持續時間與觸發原因等屬性。情緒類型採用預定義列舉值，目前支援中性、快樂、悲傷、驚訝、生氣與害羞六種基本情緒。強度值表示情緒程度，數值範圍為 0.0 至 1.0，影響動畫參數變化幅度。持續時間定義情緒維持長度，用於自動情緒衰減與恢復中性狀態。觸發原因記錄導致情緒變化之事件或訊息內容，用於除錯與分析。

動畫參數實體表示 Live2D 模型之單一控制參數，包含參數名稱、當前值、目標值、數值範圍與過渡狀態等屬性。參數名稱對應 VTS API 中之參數識別符，如 FaceAngleX 或 MouthOpen。當前值與目標值分別記錄參數之即時數值與過渡終點數值。數值範圍定義參數之最小值與最大值，用於邊界檢查與正規化。過渡狀態記錄過渡起始時間、過渡時長與 easing 函數類型，用於插值計算。

各實體間存在明確關聯關係。訊息實體與記憶實體透過來源對話識別碼關聯，一次對話可產生多筆記憶項。情緒狀態實體與訊息實體透過觸發原因關聯，追溯情緒變化之來源。情緒狀態實體與動畫參數實體透過情緒映射表關聯，特定情緒對應一組參數配置。此關聯關係設計使系統能追蹤資料血緣並進行跨模組分析。

3.2 向量資料庫設計

向量資料庫為 RAG 模組之核心儲存元件，負責語意向量之高效檢索。本節說明資料庫架構、索引策略與持久化機制。

系統採用 FAISS 作為向量資料庫引擎，選擇 IndexFlatIP 索引類型進行精確內積搜尋。此索引類型將所有向量儲存於連續記憶體空間，查詢時進行暴力搜尋，適合中小規模資料集。當記憶項數量超過一萬筆時，系統可切換至 IndexIVFFlat 索引類型，透過倒排索引結構加速檢索，犧牲少量準確度換取更快查詢速度。索引類型透過配置檔案控制，系統啟動時根據記憶項數量自動選擇適當索引。

向量維度由嵌入模型決定，jina-embeddings-v3 模型產生 1024 維向量，OpenAI text-embedding-3-small 模型產生 1536 維向量。系統於索引建構時驗證向量維度一致性，拒絕不符合維度要求之向量寫入。維度資訊儲存於索引元資料中，便於模型切換時進行相容性檢查。

資料庫採用記憶體優先策略，所有向量索引載入至記憶體以降低查詢延遲。此策略於一般硬體環境下可支援十萬筆記憶項，佔用記憶體約 400 MB 至 600 MB。當記憶項數量持續增長超過記憶體容量時，系統提供記憶清理機制，根據時效性與使用頻率移除舊記憶項。

持久化機制採用定期匯出策略，系統每隔固定時間或記憶項新增達一定數量時，將向量索引與元資料匯出至磁碟檔案。匯出檔案包含 FAISS 索引二進位檔案與 JSON 格式元資料檔案，前者儲存向量與索引結構，後者儲存記憶內容、時間戳記與權重資訊。系統啟動時自動偵測匯出檔案存在性，若存在則載入歷史記憶，否則建立空白索引。

資料庫設計亦考量備份與恢復需求。系統支援手動觸發記憶匯出與匯入操作，使用者可將記憶庫備份至外部儲存並於需要時恢復。匯出格式採用版本標籤，確保不同版本系統間之相容性。當版本不相容時，系統提供轉換工具協助格式升級。

3.3 記憶模組資料結構

記憶模組管理兩類記憶資料：短期對話記憶與長期語意記憶。本節說明兩類記憶之資料結構與管理策略。

短期對話記憶採用佇列結構，儲存最近 N 輪對話之訊息內容。佇列長度透過配置參數設定，預設值為 10 輪對話即 20 則訊息。當新訊息加入時，若佇列已滿則移除最舊訊息，維持固定長度。短期記憶用於 LLM 推理時之上下文組裝，確保模型能理解近期對話脈絡。

短期記憶項目包含完整訊息實體，儲存角色、內容、時間戳記與平台資訊。系統不對短期記憶執行向量化或檢索操作，直接以原始文字形式傳遞至 LLM。此設計降低處理開銷並保留訊息完整資訊。短期記憶於程序記憶體中維護，不進行持久化儲存。

長期語意記憶採用向量索引結構，儲存所有歷史對話之語意表示。每次對話結束後，系統將對話片段轉換為語意向量並寫入記憶庫。長期記憶用於語境檢索與知識回補，當使用者提問涉及歷史資訊時，系統透過語意檢索擷取相關記憶片段。

長期記憶項目包含記憶實體之完整屬性，向量資料儲存於 FAISS 索引，元資料儲存於關聯字典結構。記憶識別碼作為向量索引與元資料字典之關聯鍵，確保資料同步。系統於查詢時先透過 FAISS 檢索相似向量並取得識別碼列表，再透過識別碼查詢元資料字典取得完整記憶內容。

記憶權重資訊儲存於獨立字典結構，以記憶識別碼為鍵，權重數值為值。權重更新操作頻繁執行，獨立儲存避免影響向量索引結構。權重字典於持久化時與元資料一併匯出，確保權重資訊不遺失。

記憶模組亦管理記憶項之生命週期。系統定期檢查記憶項之最後存取時間，超過閾值且權重低於下限之記憶項標記為待刪除。待刪除記憶項於下次持久化時從索引中移除，釋放儲存空間。此自動清理機制維持記憶庫之語意新鮮度並控制資料規模。

3.4 資料流設計

資料流設計說明資料於系統各模組間之傳遞路徑、轉換過程與處理邏輯。本節描述文字互動與語音互動兩種典型資料流。

文字互動資料流始於平台介面模組接收使用者文字輸入。訊息資料包含原始文字內容與平台元資訊，傳遞至中央控制模組進行預處理。預處理包括文字正規化、敏感詞過濾與格式驗證。處理後訊息加入短期記憶佇列，並觸發記憶檢索流程。

RAG 模組接收訊息文字，透過嵌入模型轉換為語意向量，於向量資料庫中執行相似度檢索。檢索結果包含相關記憶項之內容文字與相似度分數，經權重計算與機率選擇後產生最終檢索片段。檢索片段與短期記憶一併組裝為 LLM 提示內容。

LLM 模組接收完整提示，執行模型推理並生成回應。回應內容經解析為文字部分與結構化部分，文字部分傳遞至輸出模組，結構化部分包含情緒標籤與動作指令傳遞至 VTS 模組。回應文字同時加入短期記憶佇列與長期記憶庫，完成一次互動循環。

語音互動資料流擴展文字互動流程，於輸入與輸出階段增加語音處理步驟。使用者語音訊號經由麥克風輸入至 STT 模組，轉換為文字後進入與文字互動相同之處理流程。LLM 生成回應文字後，傳遞至 TTS 模組執行語音合成。合成音訊透過音訊輸出裝置播放，同時情緒與動作資訊傳遞至 VTS 模組驅動虛擬角色動畫。

資料流設計採用非同步管道模式，各處理階段以協程形式執行，透過訊息佇列傳遞中間結果。此設計使各階段並行處理並提升整體吞吐量。例如當 TTS 模組執行語音合成時，RAG 模組可同時處理下一則使用者訊息之記憶檢索。管道式設計亦便於效能監控與瓶頸識別，各階段處理時間獨立測量並記錄至日誌。

資料流中各模組間傳遞之資料格式均採用結構化物件，避免使用字串或字典等弱型別結構。結構化物件定義明確之屬性與型別，透過型別檢查確保資料完整性。此設計降低模組間耦合並提升程式碼可讀性與可維護性。

綜上所述，資料設計為系統實作奠定穩固基礎，定義清晰之資料模型、高效之儲存結構與流暢之資料流，確保系統能穩定處理複雜之多模態互動流程。

第4章 介面設計

本章說明 Norgo 系統之介面設計，定義模組間資料傳遞結構、介面契約與事件驅動機制。介面設計為系統模組化之關鍵，確保各模組透過明確定義之資料結構與方法進行協作，維持低耦合與高內聚特性。

4.1 模組間資料傳遞結構

系統定義六種核心資料類別作為模組間通訊之標準格式，各類別採用 Python dataclass 實作以確保型別安全與欄位明確性。

Platform Message 封裝來自不同平台之使用者輸入，包含 platform（平台識別，如 "discord"、"telegram"）、user_id（使用者唯一識別碼）、user_name（顯示名稱）、content（文字內容）、audio_data（語音訊息之原始音訊位元組，純文字訊息為 None）、channel_id（頻道識別碼）與 timestamp（訊息時間戳記）七個欄位。此結構使中央控制模組無需關心訊息來源平台之差異，統一處理所有輸入。欄位定義如表 4.1 所示。

欄位名稱	資料型別	必填	說明	範例值
platform	str	是	平台識別碼	"discord"
user_id	str	是	使用者唯一識別碼	"123456789"
user_name	str	是	使用者顯示名稱	"小明"
content	str	是	文字內容	"今天天氣如何？"
audio_data	bytes | None	否	語音訊息之原始音訊	None
channel_id	str	是	頻道識別碼	"987654321"
timestamp	datetime	是	訊息時間戳記	2025-01-15T14:30:00Z

表 4.1 Platform Message 欄位定義

Transcript Result 封裝語音辨識模組之輸出，包含 text（完整辨識文字）、language（偵測或指定之語言代碼）與 segments（片段列表，每個片段含 start、end、text）三個欄位。segments 欄位支援字幕生成與時間對齊需求，confidence 欄位供後續模組判斷是否需要使用者確認。欄位定義如表 4.2 所示。

欄位名稱	資料型別	必填	說明	範例值
text	str	是	完整辨識文字	"請問你是誰?"
language	str	是	偵測或指定之語言代碼	"zh"

segments	List[Segment]	是	片段列表	[{"start": 0.0, "end": 1.2, "text": "請問"}]

表 4.2 Transcript Result 欄位定義

Memory Item 封裝檢索增強模組回傳之記憶項，包含 memory_id（唯一識別碼）、content（記憶文字內容）、similarity（與查詢之語意相似度，0-1）、weight（歷史權重值）與 timestamp（記憶建立時間）五個欄位。retrieve 方法回傳 Memory Item 列表，依 similarity 降序排列。欄位定義如表 4.3 所示。

欄位名稱	資料型別	必填	說明	範例值
memory_id	str	是	唯一識別碼	"mem_a1b2c3d4"
content	str	是	記憶文字內容	"使用者喜歡晴天"
similarity	float	是	與查詢之語意相似度	0.87
weight	float	是	歷史權重值	1.23
timestamp	datetime	是	記憶建立時間	2025-01-10T09:15:00Z

表 4.3 Memory Item 欄位定義

LLM Response 封裝語言模型模組之輸出，包含 text（回應文字內容）、emotion（情緒標籤，如 "happy"、"sad"、"neutral"）、actions（動作指令列表，如 ["wave", "blush"]）三個欄位。當模型啟用 JSON 模式時，模組自動解析輸出並填入對應欄位；純文字模式時 emotion 預設為 "neutral"，actions 為空列表。欄位定義如表 4.4 所示。

欄位名稱	資料型別	必填	說明	範例值
text	str	是	回應文字內容	"今天台北氣溫約 25 度"
emotion	str	是	情緒標籤	"happy"
actions	List[str]	是	動作指令列表	["smile", "nod"]

表 4.4 LLM Response 欄位定義

4.2 模組介面契約

各核心模組定義抽象基礎類別，聲明對外提供之方法簽名與預期行為，具體實作類別繼承抽象類別並實作所有方法。介面設計遵循依賴反轉原則，上層模組依賴抽象介面而非具體實作，使模組可獨立替換而不影響呼叫方。

語音辨識模組介面 BaseSTT 定義 transcribe 方法，接收 audio（音訊資料，numpy.ndarray 或檔案路徑）與 language（可選，語言代碼）兩個參數，回傳 Transcript

Result。非同步版本 transcribe_async 採用相同參數並回傳協程。實作類別需處理音訊格式轉換與取樣率調整，確保輸入符合底層模型要求。

語言模型模組介面 BaseLLM 定義 generate 方法，接收 messages（對話訊息列表，每則含 role 與 content）、context（RAG 檢索之記憶片段列表）與 json_mode（是否啟用結構化輸出）三個參數，回傳 LLM Response。串流版本 generate_stream 回傳非同步迭代器，逐步產出部分回應文字。實作類別負責組裝完整提示、呼叫底層模型並解析輸出格式。

檢索增強模組介面 BaseRAG 定義三個核心方法。add_memory 接收 content（文字內容）與 metadata（可選，額外標籤）兩個參數，將記憶項寫入向量資料庫並回傳 memory_id。retrieve 接收 query（查詢文字）與 top_k（回傳數量，預設 5）兩個參數，回傳 Memory Item 列表。update_weights 接收 retrieved_items（本次檢索結果）與 selected_id（實際使用之記憶項識別碼）兩個參數，執行權重更新演算法。

語音合成模組介面 BaseTTS 定義 synthesize 方法，接收 text（待合成文字）、emotion（情緒類型，影響韻律）與 speed（語速倍率，預設 1.0）三個參數，回傳 AudioData。synthesize_to_file 額外接收 file_path 參數，直接將音訊寫入檔案。實作類別需處理文字正規化與情緒參數映射。

虛擬角色控制模組介面 BaseVTS 定義 set_emotion 方法，接收 EmotionCommand 物件並驅動虛擬角色展現對應表情。trigger_action 接收 action_id（動作識別碼，對應 VTS Hotkey）觸發瞬時動作。get_available_parameters 回傳當前模型支援之參數列表，供初始化時建立映射表。所有方法皆為非同步實作以配合 WebSocket 通訊。

模組介面採用統一錯誤處理策略，定義 ModuleError 基礎異常類別，各模組衍生具體異常如 STTError、LLMError、RAGError 等。異常物件包含 error_code（錯誤碼）、message（錯誤訊息）與 recoverable（是否可重試）三個屬性，呼叫方根據 recoverable 決定重試或降級處理。

4.3 事件驅動機制

系統採用事件驅動架構協調模組間協作，各模組透過發布與訂閱事件進行通訊，避免直接依賴。事件物件包含 event_type（事件類型字串）、timestamp（事件產生時間）與 payload（事件資料，對應前述資料類別）三個屬性。

系統定義六種核心事件類型。MessageReceived 於平台介面模組接收使用者輸入時發布，payload 為 PlatformMessage。SpeechRecognized 於語音辨識完成時發布，payload 為 TranscriptResult。MemoryRetrieved 於記憶檢索完成時發布，payload 為 MemoryItem 列表。ResponseGenerated 於語言模型生成完成時發布，payload 為 LLMResponse。SpeechSynthesized 於語音合成完成時發布，payload 為 AudioData。EmotionChanged 於情緒狀態變化時發布，payload 為 EmotionCommand。表 4.9 定義系統核心事件類型與對應之 payload 結構。

事件類型	觸發時機	發布者	主要訂閱者
MessageReceived	平台接收使用者輸入	PlatformInterface	CoreController
SpeechRecognized	語音辨識完成	STTModule	CoreController
MemoryRetrieved	記憶檢索完成	RAGModule	LLMModule
ResponseGenerated	語言模型生成完成	LLMModule	TTSModule, VTSModule
SpeechSynthesized	語音合成完成	TTSModule	PlatformInterface
EmotionChanged	情緒狀態變化	CoreController	VTSModule

表 4.5 系統事件類型定義

事件註冊採用裝飾器模式，開發者透過 @event_listener(event_type, priority) 裝飾器標註處理函式，priority 參數控制執行順序，數值越小優先級越高。系統於初始化階段掃描所有標註函式並建立映射表。事件分發器接收事件後，依優先級順序透過 asyncio.gather 併發執行所有已註冊之處理函式，單一處理函式之異常不影響其他函式執行。

典型互動流程中，事件依序觸發形成處理鏈：MessageReceived 觸發中央控制模組判斷輸入類型，若為語音則呼叫 STT 模組並發布 SpeechRecognized，隨後 RAG 模組執行檢索並發布 MemoryRetrieved，LLM 模組生成回應並發布 ResponseGenerated，最後 TTS 模組與 VTS 模組分別訂閱 ResponseGenerated 事件並併發執行語音合成與表情控制。此事件驅動設計使各模組保持獨立，新增功能僅需定義新事件類型並註冊處理函式，無需修改既有程式碼。

參考文獻

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020. arXiv:2005.11401.

[2] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," in Proc. ICML, 2023. arXiv:2212.04356.

[3] J. Johnson, M. Douze, and H. Jégou, "Billion-scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535-547, 2021.

[4] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The Faiss library," arXiv:2401.08281, 2024.

[5] C.-J. Hsu, Y.-C. Lin, C.-C. Lin, W.-C. Chen, H. L. Chung, C.-A. Li, et al., "BreezyVoice: Adapting TTS for Taiwanese Mandarin with Enhanced Polyphone Disambiguation -- Challenges and Insights," arXiv:2501.17790, 2025.

[6] E. Casanova, K. Shih, E. Gölge, N. Camp, Y. Gao, and the Coqui TTS Team, "XTTS: a Massively Multilingual Zero-Shot Text-to-Speech Model," arXiv:2406.04904, 2024.

[7] E. Gölge, "XTTS v2: Massively Multilingual Zero-Shot Text-to-Speech," Coqui AI Blog, 2023. [Online]. Available: https://docs.coqui.ai/en/latest/models/xtts.html

[8] JinaAI, "jina-embeddings-v3: Multilingual Embeddings with Task LoRA," 2024. [Online]. Available: https://huggingface.co/jinaai/jina-embeddings-v3

[9] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural Discrete Representation Learning," in Proc. NeurIPS, 2017. arXiv:1711.00937.

[10] J. Kong, J. Kim, and J. Bae, "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis," in Proc. NeurIPS, 2020.

[11] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.

[12] AI @ Meta, "The Llama 3 Herd of Models," arXiv:2407.21783, 2024.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Proc. NeurIPS, 2017, pp. 5998-6008.